

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/335788583>

A Survey of DevOps Concepts and Challenges

Preprint · September 2019

DOI: 10.48550/arXiv.1909.05409

CITATIONS

0

READS

3,560

5 authors, including:



Leonardo Alexandre Ferreira Leite
University of São Paulo

35 PUBLICATIONS 679 CITATIONS

SEE PROFILE



Carla Rocha
University of Brasília

24 PUBLICATIONS 595 CITATIONS

SEE PROFILE



Fabio Kon
University of São Paulo

362 PUBLICATIONS 5,720 CITATIONS

SEE PROFILE



Dejan Milojicic
Hewlett Packard Enterprise

287 PUBLICATIONS 7,799 CITATIONS

SEE PROFILE

A Survey of DevOps Concepts and Challenges

LEONARDO LEITE, University of São Paulo, Brazil

CARLA ROCHA, University of Brasília, Brazil

FABIO KON, University of São Paulo, Brazil

DEJAN MILOJICIC, Hewlett Packard Labs, Palo Alto, USA

PAULO MEIRELLES, Federal University of São Paulo, Brazil

DevOps is a collaborative and multidisciplinary organizational effort to automate continuous delivery of new software updates while guaranteeing their correctness and reliability. The present survey investigates and discusses DevOps challenges from the perspective of engineers, managers, and researchers. We review the literature and develop a DevOps conceptual map, correlating the DevOps automation tools with these concepts. We then discuss their practical implications for engineers, managers, and researchers. Finally, we critically explore some of the most relevant DevOps challenges reported by the literature.

CCS Concepts: • **Software and its engineering** → **Software development process management; Programming teams; Software post-development issues;**

Additional Key Words and Phrases: DevOps, continuous (delivery, deployment, integration), release process, versioning, configuration management, and build process

ACM Reference Format:

Leonardo Leite, Carla Rocha, Fabio Kon, Dejan Milojicic, and Paulo Meirelles. 2019. A Survey of DevOps Concepts and Challenges. *ACM Comput. Surv.* 1, 1, Article 1 (January 2019), 35 pages. <https://doi.org/10.1145/3359981>

1 INTRODUCTION

Even though the DevOps movement has been discussed for nearly a decade, it lacks a widely accepted definition. By consolidating the most cited definitions of DevOps, we crafted our definition, similar to the one proposed by Dyck *et al.* [16], which we adopt throughout this paper:

DevOps is a collaborative and multidisciplinary effort within an organization to automate continuous delivery of new software versions, while guaranteeing their correctness and reliability.

Desiring to improve their delivery process [121], enterprises are widely adopting DevOps [7, 18, 20]. Although in discordance with most of the academic definitions, the software industry also uses the word “DevOps” to describe a well-paid job title [23][113]. Becoming a DevOps engineer is an attractive opportunity for software professionals. DevOps is also an important phenomenon studied by software engineering researchers and already a mandatory topic in software engineering courses.

Authors’ addresses: Leonardo Leite, University of São Paulo, São Paulo, Brazil, leofl@ime.usp.br; Carla Rocha, University of Brasília, Brasília, Brazil, caguiar@unb.br; Fabio Kon, University of São Paulo, São Paulo, Brazil, kon@ime.usp.br; Dejan Milojicic, Hewlett Packard Labs, Palo Alto, Palo Alto, USA, dejan.milojicic@hpe.com; Paulo Meirelles, Federal University of São Paulo, São Paulo, Brazil, paulo.meirelles@unifesp.br.

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

© 2019 Association for Computing Machinery.

0360-0300/2019/1-ART1 \$15.00

<https://doi.org/10.1145/3359981>

DevOps and its **challenges** can be discussed from three perspectives: engineers, managers, and researchers. **Engineers** benefit from (1) qualifying themselves for a DevOps position, a technically hard task guided by over 200 papers, 230 books, and 100 tools [63]. Engineers also need to (2) learn how to re-architect their systems to embrace continuous delivery. **Managers** want to know (3) how to introduce DevOps into an organization and (4) how to assess the quality of already-adopted DevOps practices. Managers and engineers, also referred to as *practitioners*, share the necessity of choosing the best automation toolset. Finally, academic **researchers** (5) conduct studies to determine the state of practice in DevOps, thereby contributing to discussions among engineers and managers, and (6) educate a new generation of software engineers on DevOps principles and practices.

In this context, our *research problem* is devising a conceptual framework to guide engineers, managers, and academics in the exploration of DevOps tools, implications, and challenges. There are several studies and a few surveys tackling DevOps challenges. However, with few exceptions [33], they focus on a single perspective to address a given problem. In this context, this survey contributes to the field by investigating and discussing DevOps concepts and challenges from multiple perspectives: engineers, managers, and researchers. Our review also explores a much broader range of sources and is more up-to-date than previous studies. We exploit technical implication and complexity of adopting DevOps such as automation, tightly versus loosely coupled architectures, containerization versus configuration management to deployment automation, and toolset management. Previous surveys did not cover these practical aspects and implications of DevOps.

We first survey and analyze an academic bibliography. We explain the selection and analysis procedures of these works in Section 3. The selected studies are categorized and described in Section 4. Then, we complement our discussions with non-academic sources, such as books and posts from practitioners' blogs; we describe these other sources in Section 4. Also in this section, we compare our work to other reviews on DevOps.

Based on the reviewed literature, Section 5 presents the construction of a conceptual framework [100] on DevOps composed of five conceptual maps. We relate these concepts to the engineer and manager perspectives. We also look at the practical side of DevOps by analyzing the DevOps tools under the perspective of the concepts in Section 6. We categorize these tools, correlate them to concepts, and discuss which roles in the organization should use which tools.

The discussion of DevOps concepts and tools lays the basis to raise essential implications for engineers, managers, and researchers, which we present in Section 7. Although most of the implications are straightforward, some challenges are not entirely illuminated by the current literature. We summarize and discuss some of the main DevOps challenges in Section 8, debating limitations and even contradictions found in the literature.

We close with the limitations of this survey in Section 9 and with concluding remarks in Section 10. In the next section, we briefly introduce DevOps history, motivations, and goals.

2 DEVOPS

DevOps is an evolution of the agile movement [8]. Agile Software Development advocates small release iterations with customer reviews. It assumes the team can release software frequently in some production-like environment. However, pioneer Agile literature puts little emphasis on deployment-specific practices. No Extreme Programming (XP) practice, for example, is about deployment [57]. The same sparse attention to deployment is evident in traditional software engineering processes [93] and books [106, 116]. Consequently, the transition to production tends to be a stressful process in organizations, containing manual, error-prone activities, and, even, last-minute corrections [83]. DevOps proposes a complementary set of agile practices to enable the iterative delivery of software in short cycles effectively.

From an organizational perspective, the DevOps movement promotes closer collaboration between developers and operators. The existence of distinct silos for operations and development is still prevalent: operations staff are responsible for managing software modifications in production and for service levels [48]; development teams, on the other hand, are accountable for continuously developing new features to meet business requirements. Each one of these departments has its independent processes, tools, and knowledge bases. The interface between them in the pre-DevOps era was usually a ticket system: development teams demanded the deployment of new software versions, and the operations staff manually managed those tickets.

In such an arrangement, development teams continuously seek to push new versions into production, while operations staff attempt to block these changes to maintain software stability and other non-functional concerns. Theoretically, this structure provides higher stability for software in production. However, in practice, it also results in long delays between code updates and deployment, as well as ineffective problem-solving processes, in which organizational silos blame each other for production problems.

Conflicts between developers and operators, significant deployment times, and the need for frequent and reliable releases led to inefficient execution of agile processes. In this context, developers and operators began collaborating within enterprises to address this gap. This movement was coined “DevOps” in 2008 [68].

In the *Continuous Delivery* book [83], Humble advocates for an *automated deployment pipeline*, in which any software version committed to the repository must be a production-candidate version. After passing through stages, such as compilation and automated tests, the software is sent to production by the press of a button. This process is called *Continuous Delivery*. A variant is the *continuous deployment* [81], which automatically sends to production every version that passes through the pipeline. Many authors closely relate DevOps to continuous delivery and deployment [42, 46, 47, 49]. Yasar, for example, calls the deployment pipeline a “DevOps platform” [49].

Besides automating the delivery process, DevOps initiatives have also focused on using automated runtime monitoring for improving software runtime properties, such as performance, scalability, availability, and resilience [2, 3, 8, 40]. From this perspective, “Site Reliability Engineering” [58] emerged as a new term related to DevOps work at runtime.

3 STUDY DESIGN

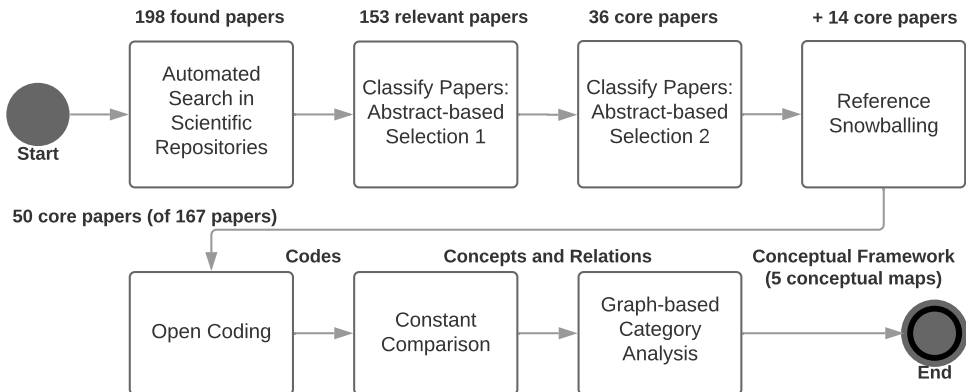


Fig. 1. Sequence of steps taken in our study

The recent growth of the DevOps community prompted a rapid increase in publications, imposing a barrier to a thorough analysis of every published work on the matter. We adopted a qualitative procedure to limit the number of analyzed studies and the information extracted from them. We established a search and selection protocol, inspired by the procedures of systematic literature reviews [62], and set analysis procedures based on Grounded Theory strategies [64]. Our primary outcome is the construction of a conceptual framework, composed of a set of conceptual maps that describe the main concepts of DevOps and how they relate to each other (Section 5). In this section, we detail our search and selection protocol, as well as our procedures to build our conceptual maps. This sequence is outlined in Figure 1.

3.1 Search and selection protocol

We searched for articles in major scientific papers repositories, namely: ACM Digital Library; IEEE Xplore Digital Library; the Spring Link. We also explored Google Scholar, but its outcomes did not provide additional relevant results. The search was conducted according to the following criteria:

- Title or abstract must contain the word “DevOps.”
- Papers must be published in journals, conferences, congresses, or symposiums (workshop papers were not considered).
- Papers must be written in English.
- Papers must have at least three pages.
- Literature reviews were not included. We present them as related work in Section 4.2.

This search resulted in 198 papers. We classified these papers into the following categories: irrelevant, relevant, and core papers. To perform the classification process, we read the abstract of all 198 papers and, in some cases, read their content too. Irrelevant papers used the keyword DevOps only for their initial background, not being indeed papers about DevOps. Using this criterion, we discarded 45 irrelevant papers, reducing to 153 the number of relevant papers to be analyzed. Then, we applied a second filter employing another set of criteria to identify papers that would be unlikely to generate any further concepts, handling theoretical saturation [65]. For example, we identified papers with overlapping topics, that would only reinforce already existing concepts. We then proceeded with our selection process and, from the 153 relevant papers, we obtained a subset of 36 core papers, which were thoroughly read. The remaining 117 ones were classified just as relevant papers.

To reduced the subjectivity of this selection process, at least two authors had to agree on each outcome; the authors produced their classifications independently and when there were different classifications, each paper was individually examined until a consensus was reached. Moreover, even though there were no hard rules for the selection process, we followed some guidelines:

- If a paper discusses DevOps peculiarities or challenges in some specific context, it is a core paper.
- If a paper is primarily centered on a single tool and does not discuss DevOps itself, it is classified as relevant.
- If a paper has an extended or more cited version, only the most relevant version is a core paper, whereas the other one is just a relevant paper.
- If several papers are about the same subject, only some of them are selected as core papers, whereas the others are classified as relevant.
- If a paper is about applying DevOps in a specific context, but does not elaborate beyond the state-of-the-art, it is discarded.
- If a paper discusses some topic (e.g., big data) and uses DevOps only as background, it is discarded.

- If a paper only presents a simple proposal or opinion, without validation, it is discarded.
- In exceptional circumstances, studies become core papers without following all the above rules if at least three authors agree.

We also applied a snowballing process [123] to cover important work not found with the query string “devops”. We explored historical, seminal, highly-cited, and recent references. From this criterion, we considered 14 additional core papers (i.e., equaling 167 relevant papers with 50 core papers).

We list all the 50 core papers on Page 31 at the end of this paper. We identify each paper with a unique reference code, composed of a sequence number and a letter indicating how we found the article: “A” for the ACM Digital Library; “I” for the IEEE Xplore Digital Library; “S” for Springer Link; and “B” for the snowballing process. This reference code is later used to display the papers on the conceptual maps.

Tools like Mendeley, Libre Office spreadsheets, SQLite databases, and some Python scripts, like the ones that generated our bar plots using Pandas and Pyplot.

3.2 Producing the conceptual framework

According to Miles and Huberman, a conceptual framework explains graphically the main things to be studied in a given field, including elements such as key factors and their relationships [100]. Our conceptual framework is composed of conceptual maps, which are diagrams structured as graphs in which nodes depict concepts and arrows represent relationships among concepts [77]. Our process of building the conceptual maps was based on Grounded Theory, which is a recognized method for conducting emergent qualitative research [66] and whose adoption in software engineering has grown considerably in recent years [118]. According to Charmaz, it reduces preconceived ideas about the research problem and helps researchers remain amenable to alternative interpretations and apprehend the data in different ways [64]. Grounded Theory is usually applied to primary sources, such as interviews and observational field-notes [112]. In this paper, we adopted Grounded Theory strategies in the literature review to handle the inherent subjectivity of conceptual analysis.

A core strategy of Grounded Theory is *coding* [64], which breaks down and labels data into smaller components [112]. In the *initial coding*, also called *open coding*, researchers avoid injecting previous assumptions, biases, and motivations [118]. Researchers then apply *constant comparison* by constantly comparing and correlating codes from different sources to produce *concepts* and group concepts into *categories* [112, 118]. In our study, we read the 50 core studies and applied *open coding* on them to produce codes. Then, through *constant comparison*, we analyzed, abstracted, and grouped the codes into concepts and categories, presenting each category in a conceptual map.

The open coding phase was carried as follows: while reading a core paper, the researchers used the Mendeley software to highlight excerpts containing potential DevOps concepts. In the following, we provide six examples of codes and their corresponding highlighted text.

DevOps emphasizes collaboration: “DevOps is a new phenomenon in software engineering, emphasizing collaboration (...) that bridge software development and operations activities (...)” [31].

DevOps advocates collaboration: “DevOps is a set of practices that advocate the collaboration between software developers and IT operations (...)” [10].

DevOps emphasizes collaboration and communication: “DevOps is a culture, movement or practice that emphasizes the collaboration and communication of both software developers and Information Technology (IT) professionals (...)” [9].

DevOps shares knowledge and tools: “DevOps was introduced to include cross-functional roles of teams with four perspectives: (...), and 4) sharing of knowledge and tools (...)” [9].

DevOps has a culture of collaboration / Collaboration means sharing knowledge and tools: “Achieving these benefits within enterprises requires the discipline of DevOps: a culture of collaboration between all team members; (...); sharing of knowledge and tools (...)” [22].

DevOps emphasizes working together / working together by sharing tools, processes, and practices: “That is why the DevOps movement’s recent emergence is so heartening. It emphasizes development and operations staff working together as early as possible – sharing tools, processes, and practices to smooth the path to production (...)” [48].

By applying constant comparison in these codes coming from different sources, we defined the concept “Culture of collaboration” and linked it to the “DevOps” concept in the form “(DevOps) → is a → (culture of collaboration),” with concepts represented within parenthesis. From the excerpts we also understood that a “culture of collaboration” means sharing “knowledge, tools, processes, and practices”, so we grouped these last concepts, from different sources, to state that “(DevOps) → is a → (culture of collaboration) → based on sharing → (knowledge, tools, processes, and practices),” as we can read from Figure 6 in Section 5.

As concepts emerged, we assembled a single conceptual map, linking concepts nodes in a graph. The categories emerged afterwards mostly from observing the following connectivity patterns in the graph: *i) hubs*: a single concept is linked to many other concepts, as it is the case of “teams” in Figure 6, which is linked to another 7 concept nodes, with 13 references supporting such connections; and *ii) cycles in the corresponding undirected graph*, such as the one present in Figure 6: “(DevOps) → is a → (culture of collaboration) → shared across different types of → (teams) → may or may not have a → (DevOps role) → requires certain → (knowledge, skills, and capabilities for DevOps) ← must teach ← (programming education) ← imposes challenges for ← (DevOps).” The presented hub and cycle examples led to the emergence of the *people category*. We were driven by data to identify the following categories: *process, people, delivery, and runtime*. We then fit other concepts into these categories. If some concepts did not fit in the found categories, new categories could be created, but this did not happen.

The process of building our conceptual maps also obeyed the following rules and restrictions:

- Nodes in the map contain DevOps concepts.
- The map can link nodes to describe relations among concepts.
- A link can be read as a sentence binding the linked concepts, such as “(DevOps) → is a → (culture of collaboration).”
- Each core paper received a unique reference code.
- Every concept and link must be supported by at least one core paper.
- The link’s label contains reference codes.
- A link can have different labels, each one with its references.
- Some concepts, with common links, are grouped to save space in the figure.
- A concept can belong to multiple categories.

Usually Grounded Theory procedures end with *theoretical coding* leading to the formulation of a new theory by the development of hypotheses linking concepts and categories [118]. In this work, we do not aim at the formulation of a new DevOps theory nor to answer open questions posed by the literature. Instead, we use our conceptual framework on DevOps, built with open coding and constant comparison, as input to: *i) study and classify DevOps tools*; *ii) identify practical DevOps implications and segment such implications for engineers, managers, and researchers*; and *iii) discuss the main DevOps open challenges*.

Moreover, by providing a global view of the DevOps field, we expect our conceptual maps to be used by engineers and managers to guide their continuing education, by teams to support reflection

and continuous improvements of their DevOps journeys, and by academics to identify research topics, influences, and implications.

In the next section, we give an overview of articles analyzed to build our conceptual maps, as well as other relevant sources of knowledge on DevOps. In Section 5, we present the conceptual maps we produced to structure a set of relevant DevOps concepts grounded in the literature.

4 SOURCES OF KNOWLEDGE

A diverse community of academics and practitioners around DevOps has been formed in recent years, and it is not very obvious where they publish their reports, research, and relevant information since there are very few DevOps-specific venues. In this section, we present the sources of knowledge with the most impact, which include peer-reviewed papers, books, talks, and others. We also categorize our core papers and provide an overview of the existing surveys of DevOps, providing a list of related readings on the subject. We extract from this list the readings the DevOps community consolidated as their references, which provides a reading guide for newcomers.

4.1 Peer-reviewed literature

This survey primarily relies on evaluating peer-reviewed papers, which is standard practice in the academic community for guaranteeing the quality and credibility of published works.

Figure 2 depicts the evolution of published works on DevOps according to the criteria defined in our study design (papers collected in September 2018). Similar to most research areas, conferences generally publish preliminary results. In contrast, journals publish more mature studies. Academic journals primarily reach an academic audience, while magazines also have practitioners as their readers.

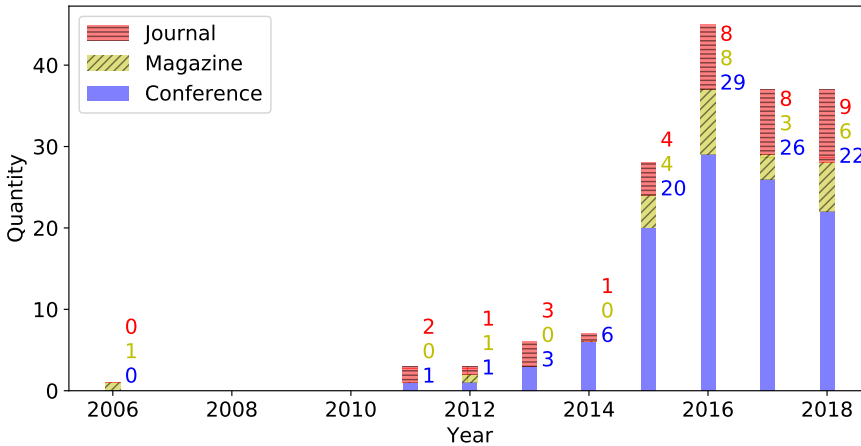


Fig. 2. Publications by source types and publication year

The “IEEE Software” magazine was a constant source of papers on DevOps, having published 13 of the 50 core papers. They are mostly relatively short papers describing real experiences faced by organizations. Therefore, it is advisable for practitioners to subscribe to this source for the latest updates on DevOps.

We did not find any peer-reviewed journal, magazine, or event dedicated exclusively to the theme. The closest related events we found were the “International Workshop on Release Engineering”

and the “International Workshop on Quality-Aware DevOps.” For researchers beginning a study on DevOps, these are two potential venues for publishing their first results. The absence of a specific venue indicates an opportunity for the academic community to organize scientific events and journal special issues dedicated to this area.

After paper selection, following the protocol presented in Section 3, we classified all 50 papers into seven categories, as shown in Table 1. The categories are described in the sequence.

Table 1. Classification of core papers

Category	Quantity	Papers
Practices	15	[3, 5, 11, 17, 19, 24, 26, 27, 30, 32, 38, 46–48, 50]
Experience report	8	[2, 6, 7, 12, 18, 20, 33, 44]
Impact	7	[25, 39–43, 49]
Concepts	7	[4, 14, 16, 21, 22, 36, 37]
Challenges	6	[10, 15, 28, 29, 31, 35]
Education	4	[1, 8, 9, 23]
Adoption	3	[13, 34, 45]

Practices: provides tools, practices, patterns, and strategies to enhance DevOps. These studies offer guidelines for selecting deployment technology [46], explanation of the use of metrics [19], how to support communication among global peers through specialized social networks [32], and how injecting infrastructure faults at production can support system reliability [3].

Experience Report: refers to reports of organizations adopting DevOps or continuous delivery/deployment. For example, Siqueira *et al.* describe the impact of a DevOps team in the context of a governmental project [44].

Impact: explores how DevOps effects other aspects of software development such as architecture [42], security [49], quality assurance [40], or the impact of DevOps on software development in specific scenarios, such as the development of software for research [12].

Concepts: introduces basic concepts in DevOps and continuous delivery, such as the deployment pipeline [22].

Challenges: covers the challenges entailed in adopting DevOps or continuous delivery/deployment. Some papers focus on specific challenges, such as DevOps in regulated domains [28], embedded systems [31], or communication [15].

Education: investigates the challenges in teaching DevOps. Articles in this category propose teaching methods [8], as well as explore the Knowledge, Skills, and Abilities (KSA) necessary for DevOps professionals [9, 23].

Adoption: covers the different models for DevOps adoption [34] and describes how an organization chooses a set of metrics to evaluate its DevOps adoption process [45]. This subject is further discussed in Section 8.2.

4.2 Related Surveys

Even though DevOps is a recent research topic, several studies have performed a Systematic Literature Review (SLR) on the subject. They aim at finding a final DevOps definition and which aspects influence its adoption on an organization. Table 2 depicts the major research questions of these previous SLR. It is worth to mention most previous SLRs performed their literature review

before 2015. It means they primarily examined the pioneering works on DevOps and did not capture the significant growth of publications depicted in Figure 2; therefore, they miss most of the currently existing literature in the topic. This is evidenced by their conclusion that initial studies have a very low quality [71].

Table 2. Primary research questions from previous DevOps SLR works

Research Question	References
RQ1: What is the meaning of the term DevOps?	[67, 71, 75, 84, 117]
RQ2: What are the issues motivating the adoption of DevOps?	[67, 71, 75]
RQ3: What are the main expected benefits of adopting DevOps?	[67, 71, 75]
RQ4: What are the main expected challenges/impediments of adopting DevOps?	[67, 71, 75, 115]

The most relevant contribution of these previous studies is to correlate the absence of a well-established DevOps definition with its effect on the perception of its benefits, expectations, and adoption challenges [71].

To overcome the lack of the initial studies quality, they deployed methods other than Systematic Literature Review to uncover both academic and practitioners perspectives and to address the novelty of DevOps. Table 3 presents a list of sources used by these other works. Our survey selected works from all cited sources and had a much broader range of analyzed works than previous SLRs. We identified 167 papers and carefully examined their titles and abstracts to select the 50 core papers, which were then extensively analyzed to provide an updated overview of DevOps.

Table 3. Sources of DevOps surveys

Author	Google Scholar	Springer Link	ACMDigital Library	IEEE Explore	Scopus	Web of Science	Others	Papers
França [67]	X						X	43
Erich [71]			X	X	X	X		27
Smeds [115]	X	X	X	X		X	X	27
Ghantous [75]	X	X	X	X			X	30
Stahl [117]					X			35
Jabbari [84]			X	X	X			49
Lwakatare [96]	X		X	X	X	X	X	22
This survey	X	X	X	X	X	X	X	167 (50)

Erich *et al.* [70, 71] is the most referenced literature review of DevOps. The authors performed an SLR [70, 71] and conducted focused interviews on DevOps principles and practices at six organizations [71]. From SLR, they extracted seven areas related to DevOps: “culture of collaboration, automation, measurement, sharing, services, quality assurance, and governance” [71]. They conclude that there was, in general, low quality on the academic studies on DevOps and a lack of studies to evidence DevOps’ effectiveness. From focused interviews, they analyzed how organizations implemented DevOps in four dimensions: governance, personal traits, department, and practitioners. They categorized answers from each organization into one of the seven areas extracted from SLR to portray their perceptions. Although both academic studies and organizations stated the benefits of adopting DevOps, Erich *et al.* concluded that no quantitative studies support such a claim [71], and reinforce the necessity of experimental studies to verify the effectiveness of DevOps.

França *et al.* [67] assumed academic studies were insufficient to address DevOps thoroughly, so they alternatively conducted a Multivocal Literature Review (MLR), in which most of the sources were extracted from gray literature, including books, websites, and industry journals and technical reports. The authors describe DevOps as a collection of principles, practices, required skills, and

organizations' motivations to adopt it. They identified seven areas related to DevOps: social aspects, automation, measurements, sharing, quality assurance, and leanness. Their main contribution was to describe DevOps characteristics associated with the practitioners' community and state-of-the-practice.

Erich *et al.* [71] and França *et al.* [67] indicate the need for more research on DevOps, especially quantitative research to evaluate DevOps' effectiveness. Even with distinct methodologies, these two works arrived at similar conclusions. They concentrate their studies on the *Process* and *People* categories of our conceptual map, to be described in Section 5. Previous surveys [67, 75, 84] did not cover the *Runtime* and *Delivery* categories of our conceptual map; they have only mentioned some of the concepts. None of the previous SLRs have discussed the technical implications and complexity of adopting DevOps practices such as automation, microservices architectures, containerization, and toolset management. The concepts and implications of practical aspects of DevOps not only empower managers to make assertive and strategic decisions but also equip engineers with best practices for DevOps adoption. We provide lessons learned of *i)* using DevOps with legacy systems; *ii)* the complexities and mistakes when adopting the microservice architecture; *iii)* guidelines for automating the delivery pipeline; *iv)* the advantages and drawbacks of teams autonomy in toolset selection. These points form the major contribution of our survey, wherein we deepen the analysis of practical aspects of DevOps adoption with the aid of a conceptual map of DevOps to guide engineers, managers, and researchers toward mastering skills necessary to enable DevOps. Additionally, the quality of studies on DevOps improved in recent years, so we have updated the implications of deploying DevOps in an organization as well as contributed to more technical implications for practitioners and researchers, such as re-designing systems architecture, deployment pipeline, and quantitative assessment metrics. Previous SLRs did not cover these latest subjects.

4.3 Books

Books are the standard source of knowledge on DevOps among practitioners. Although they usually have more pages than academic papers, they are easier to read and to assimilate. We searched Amazon.com on December 2018, for books written in English containing "DevOps" in the title. Figure 3 presents the annual distribution of a total of 238 books found in this search. Comparing to Figure 2, it is possible to see their shapes roughly shift by one year, showing how academic publication precedes book publication.

One of the seminal books on DevOps is "Continuous Delivery" [83], from 2010. It does not explicitly use the word "DevOps," but it describes in detail the deployment pipeline pattern, which is usually central to DevOps strategies. The oldest book found at Amazon containing the word "DevOps" in its title is "DevOps: High-impact Strategies" [108] from 2011. Some of the books frequently cited by our reviewed literature are "The Phoenix Project" [90] and "DevOps: A Software Architect's Perspective" [55]. The most popular books at Amazon containing the word "DevOps" in the title are "Accelerate: The Science of Lean Software and DevOps" [72], "The Phoenix Project" [90], and "The DevOps Handbook" [91].

It is worthy to note that three authors of these cited books (Nicole Forsgren, Jez Humble, and Gene Kim) also collaborate with the State of DevOps Reports [53, 121], a vital source of knowledge on DevOps, as discussed next.

4.4 Other sources

State of DevOps Reports: annual industrial research published since 2013 by renowned authors in the DevOps field based on the survey of multiple organizations worldwide [53, 121]. The reports

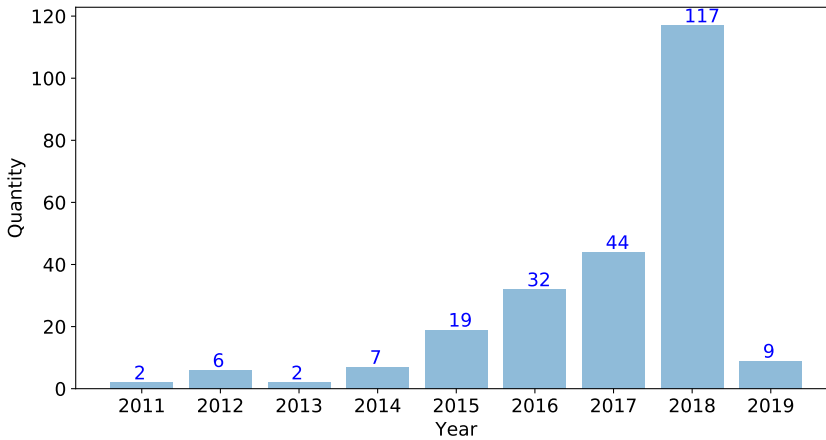


Fig. 3. Distribution of books about DevOps by publication year according to Amazon.com

provide proper usage of quantitative metrics for assessing DevOps initiatives. More about this will be discussed in Section 8.3.

The Twelve-Factor App: a manifest with architectural principles aligned with the DevOps philosophy [122]. It is a reputable source amongst practitioners and is typically used to judge architectural decisions. The present instructions like: (a) keep development, staging, and production environments as similar as possible, and (b) use environment variables to store configuration that varies across environments, among others. A related set of principles is the Reactive Manifesto, which is also used by practitioners as a guideline [59].

Talks and Videos: there are many practitioners' conferences about DevOps and conferences that include DevOps talks. Many practitioners rely on these talks for up-to-date information on DevOps, especially when the speakers are from reputable companies, such as Amazon [60], Microsoft [61], Google [120], Netflix [52], and Opscode [85]. Many of these conference talks are made available on the web, where they join many other short videos about DevOps. On Youtube or other video platforms, dozens of videos have titles matching the "What is DevOps" string.

Global community: engineers leverage global-community knowledge to overcome their daily challenges. This knowledge comes from code repositories, such as GitHub and Chef Supermarket, and from discussion forums like stackoverflow.com [32]. In this way, engineers interact not only with colleagues within the enterprise but also with peers around the world, building a global community.

5 FUNDAMENTAL CONCEPTS

In this section, we present a *conceptual framework* of fundamental concepts of DevOps. The concepts emerged from our systematic analysis of the literature, which followed the protocol presented in Section 3. Our contribution is to provide a relevant and structured set of concepts on DevOps, grounded in the literature, to support analysis and understanding of DevOps challenges.

Our conceptual framework [100] on DevOps is composed of a conceptual map outlining the conceptual categories and of other four conceptual maps that are diagrams structured as graphs in which nodes depict concepts and arrows represent relationships among concepts. There is always

at least one core bibliographic reference supporting a particular relationship. The references are represented by codes that can be identified in the List of Core Papers on page 31.

The concepts are distributed into four major categories: process, people, delivery, and runtime. The *process* category encompasses business-related concepts. *People* covers skills and concepts regarding the culture of collaboration. *Delivery* provides the concepts necessary for Continuous Delivery, and, finally, *runtime* synthesizes concepts necessary to guarantee the stability and reliability of services in a continuous delivery environment.

While the process and people categories relate more to the management perspective, runtime and delivery relate more to an engineering perspective. Moreover, while *delivery* concepts relate more to developers, *runtime* concepts relate more to the traditional operator role. The categories are depicted in Figure 4 and described in this section.

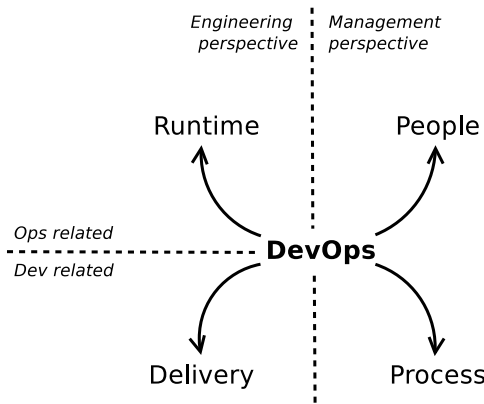


Fig. 4. DevOps overall conceptual map

Before proceeding to the categories descriptions, we briefly present some reasons to support that these categories are enough to frame the DevOps field: *i)* it seems reasonable to split “dev” and “ops” concepts as induced by the categories *delivery* and *runtime*; *ii)* it makes sense to separate more technical concepts from less technical concepts, as caused by the separation between the perspectives of engineering and management. *iii)* These categories match our DevOps definition: “a collaborative and multidisciplinary effort within an organization” regards *people*; “continuous delivery” is a *process* that is “automated” by the *delivery* techniques, which also guarantee software “correctness”; finally, software “reliability” is promoted by the *runtime* concepts.

5.1 Process

DevOps aims to achieve some business outcomes, such as reducing risk and cost, complying with regulations, and improving product quality and customer satisfaction. We grouped these concepts into the *process* category of concepts, presented in Figure 5, which reveals that DevOps achieves such business outcomes through the accomplishment of a process with frequent and reliable releases. In particular, the diagram explicit that continuous delivery leads to product quality and customer satisfaction because of the short feedback cycle it provides.

One could argue that rigorous human and hierarchical approval processes can reduce risk, comply with regulations, and provide product quality. Nevertheless, DevOps is different, once its practices are based on agile and lean principles, which embrace change and shorten the feedback cycle, as depicted in the diagram.

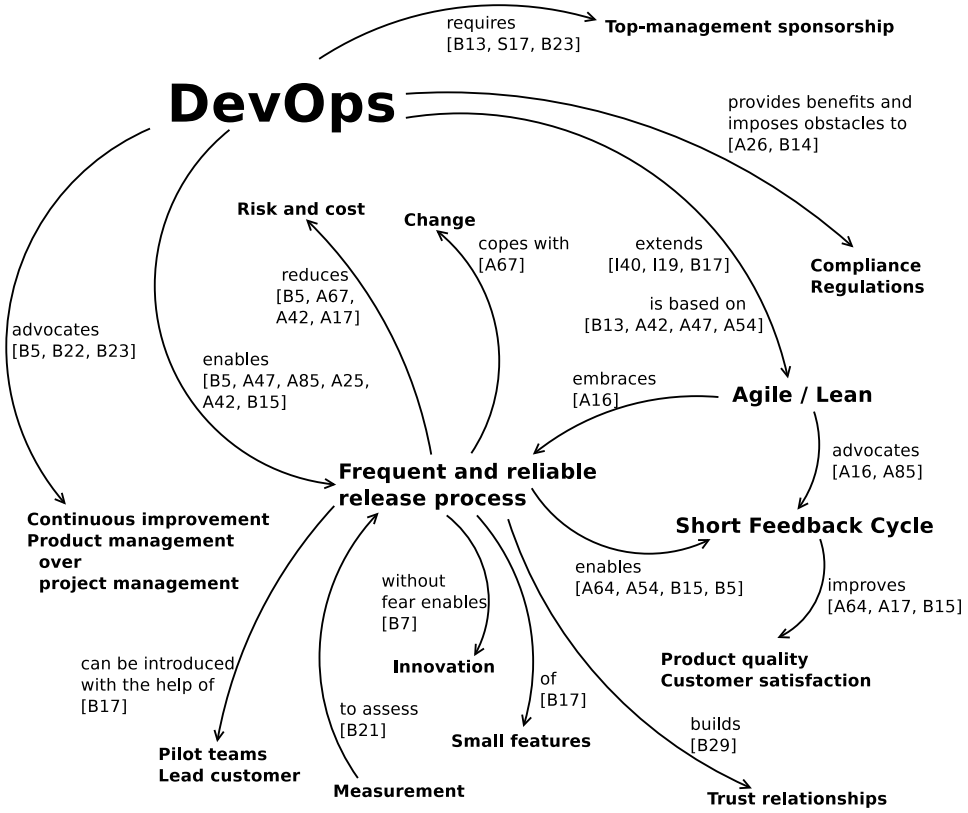


Fig. 5. Process conceptual map

5.2 People

The term “DevOps” centers on the idea of bringing together *people* from development and operations through a culture of collaboration. The concepts around this idea were grouped in the *people* category, which is presented in Figure 6. It depicts that DevOps intends to break down the walls among silos, aligning the incentives across the organization. However, breaking down silos raises many questions concerning concepts depicted in the diagram: how can organizations perform a cultural shift that implies more responsibilities for developers? How do developers acquire operations skills? Can developers and operators work in the same team and still keep different job titles? Should “DevOps” be a role? Should teams be cross-functional and include operators? Should an operator be exclusive to one team? What does it mean being an operator in the DevOps context? Although the concepts of this category are frequent in the literature, the answers to the previous questions are not clear yet, so we debate them in Section 8.2.

5.3 Delivery

The core strategy for achieving a frequent and reliable delivery process is the automation of the *deployment pipeline*, from which DevOps tools and techniques emerge. We grouped the concepts surrounding the deployment pipeline into the *delivery* category, which is presented in Figure 7. Automation tools, as depicted in the diagram, are usually open source and enable core DevOps

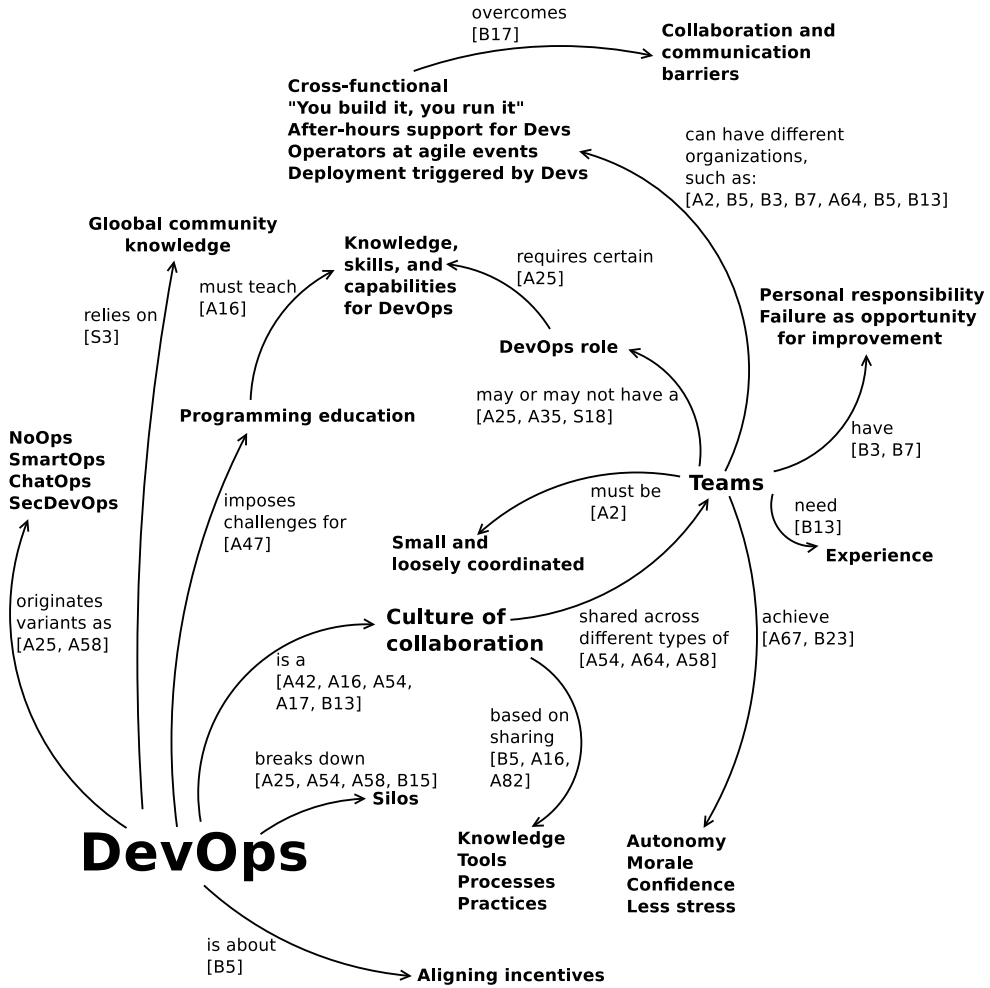


Fig. 6. People conceptual map

practices, such as versioning, testing automation, continuous integration, and configuration management. The diagram also shows the interconnection between DevOps and microservices: one supports the other, as we explore in Section 8.1. Other concepts related to microservices are also depicted, such as backward compatibility and API versioning.

There is still some debate on concrete strategies for tooling, such as the containerized approach leveraged by Docker on the one hand, and continuous configuration convergence, such as in Chef and Puppet, on the other. We discuss such debates in Section 6. However, we claim the *delivery* concepts are much more stable and accepted by the community than the *people* concepts.

5.4 Runtime

It is not enough to continuously deliver new versions, but it is also necessary for each new version to be stable and reliable. Thus, *runtime* concepts are a subsequent and necessary extension of DevOps. The *Runtime* category of concepts is presented in Figure 8, which shows the desired outcomes, such as performance, availability, scalability, resilience, and reliability. The figure also

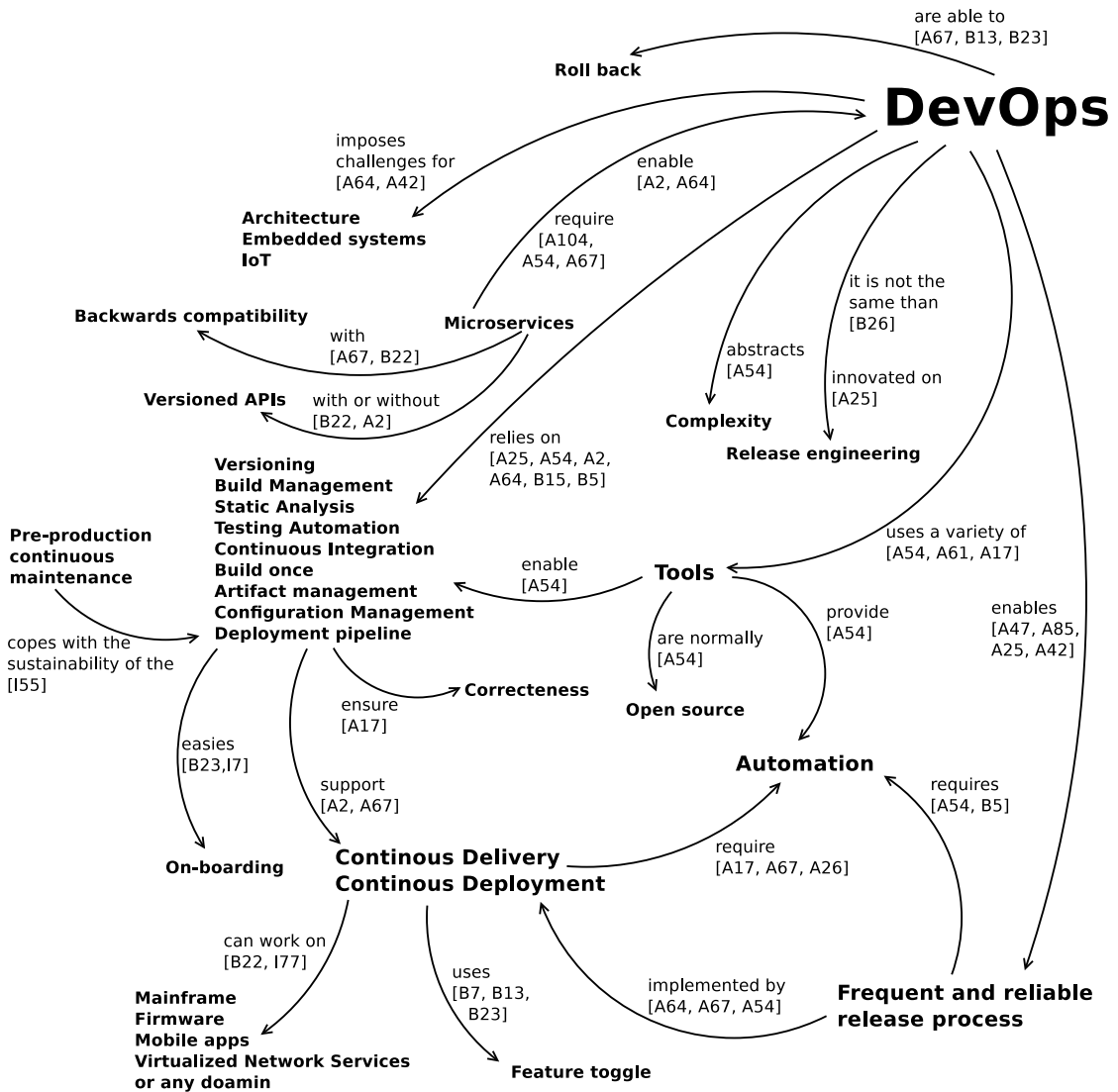


Fig. 7. Delivery conceptual map

shows paths to achieve the mentioned outcomes, like the use of infrastructure-as-code, virtualization, containerization, cloud services, and monitoring. As depicted by the diagram, DevOps can monitor high-level business metrics or low-level resource metrics. Another topic, also shown in the figure, is to run experiments in the production environment, like injecting failures to ensure software reliability, as advocated by the chaos engineering approach [3]. All of this reduces human intervention to ensure software reliability, which is another factor that challenges the traditional role of operators, as we argue in Section 8.2.

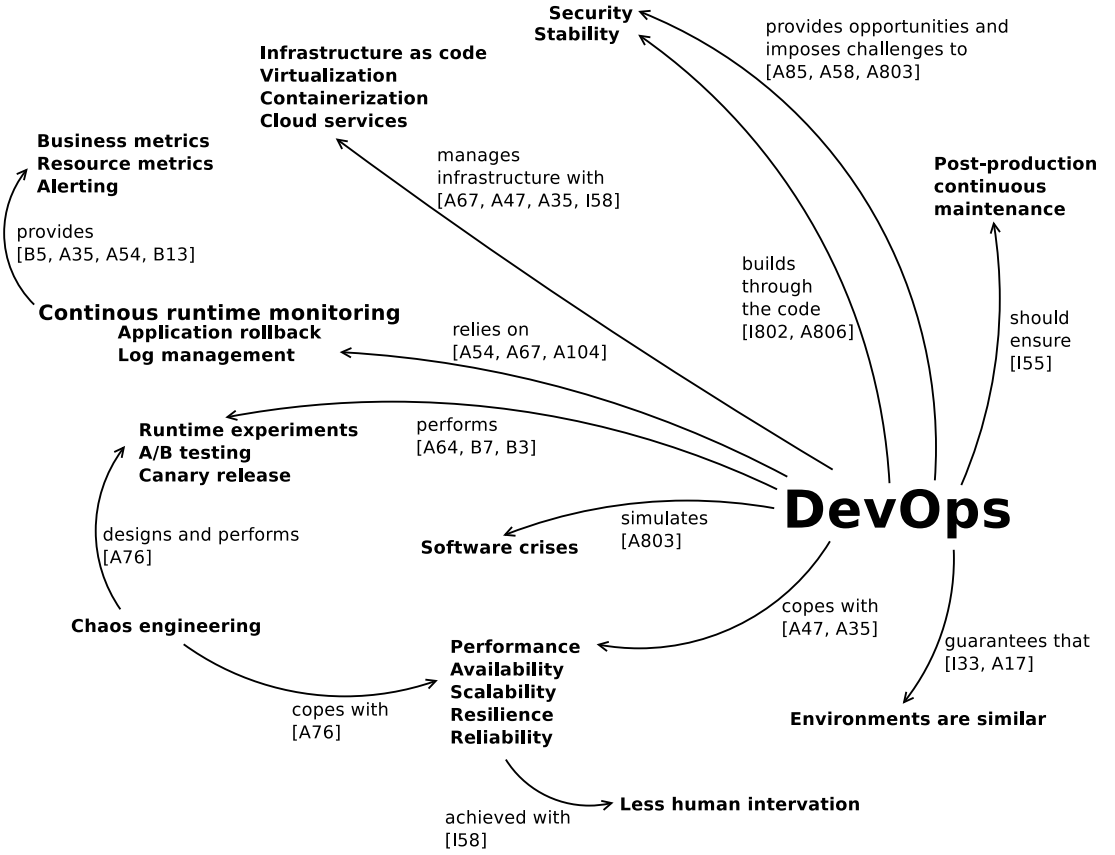


Fig. 8. Runtime category of concepts

We complement our conceptual analysis with the practical side of DevOps by investigating DevOps' tools and actors, and their relations to concepts. As the presentation of DevOps' concepts has already raised some concerns that demand further discussion, some issues will also emerge from the discussion on DevOps tools in the next section.

6 TOOLSET

We define "DevOps tools" as the tools pursuing one of the following goals: 1) assisting *human collaboration* across different departments; 2) enabling *continuous delivery*; or 3) maintaining software *reliability*. Our contribution is to associate tools to concepts presented in Section 5. This correlation between tools and concepts equips and empowers practitioners for well-informed decisions on tooling, and thereby enforces alignment between technical, managerial and organizational decisions. This conceptual approach becomes immensely important when considering the ever-changing nature of DevOps tools [42].

Although tools matter, caution is required to avoid turning them into the core of a DevOps strategy. Adopting new tools demands significant effort, and applying this effort without considering

the expected outcome can lead to considerable waste. For example, a DevOps adoption process must consider team structuring as more critical than the choice of specific tools.

DevOps heavily relies on automation tools. The choice of a proper toolset imposes challenges for DevOps professionals. It is not feasible for a single person, even an organization, to master all the DevOps tools, given the vast number of available tools [17, 27] and their rapid evolution [42].

We also contribute to discussing the most important categories of DevOps tools, presented in Table 4, by adding the following complementary information to each category: *i)* some of the most used tools in each category; *ii)* the intended users: whether developers or operators; *iii)* the goals: *human collaboration, continuous delivery or reliability*; and *iv)* DevOps concepts.

The complementary information, like tool examples, helps the reader to understand each category, leading to the following additional contributions: *i)* helping DevOps practitioners focus on organization requirements, while choosing the most suitable set of tools; *ii)* helping the reader to grasp a critical view about the objective of each tool and how to use it in an organization; *iii)* showing how the frontiers between developers and operators have been blurred in the DevOps context through toolset usage.

6.1 Tools for knowledge sharing

The DevOps strategy focuses on human collaboration across silos. Concretely, this entails different departments sharing knowledge, process, and practices, which requires sharing specific tools.

We cite GitLab and Rocket Chat as examples of tools in this category. Gitlab not only provides a code repository but also has a wiki system that enables developers and operators to share knowledge. Gitlab offers an issue management system, which is typically used by developers and business analysts to share knowledge through agile practices like issues comments and pull requests processes. Making operators also familiar with these issues helps them to understand the product and project contexts. Moreover, documenting production problems known by operators in the issue management system is an essential step toward handling non-functional requirements prioritization in the development flow. ChatOps (Chat and Operations) is a model that connects people, tools, process, and automation through conversation-driven interactions mediated by tools [103]. Rocket Chat is a communication tool that implements ChatOps by enabling a higher degree of automation and tool integration through the use of webhooks and chatbots.

These tools support *human collaboration* and the *people* category of concepts. However, despite the centrality of human collaboration in DevOps strategy, the “DevOps Periodic Table” [63], a well-known list of more than 100 DevOps tools, includes only eight tools for collaboration. This fact may suggest that human collaboration and *people* concepts have been eclipsed in the DevOps movement by the more technical goals and concepts.

6.2 Tools for source code management

Source code management tools usually intend to promote collaboration among developers. These tools are basic blocks to implement continuous integration and, therefore, *continuous delivery*. From this traditional perspective, one could relate source code management solely to the *delivery* category of concepts.

However, source code management can also be used by operators to store artifacts and automation scripts and to access software information that can impact operations activities. For example, a development bug may cause a memory leak that impacts operations. When storing infrastructure related artifacts, whether in the infrastructure-as-code style or just as plain text, operators provide developers better insight into how the software is executed. Therefore, source code sharing among developers and operators becomes a real point of collaboration.

The most traditional tools in this category are SVN and Git. More complete platforms, such as GitLab and GitHub, can also wrap Git, providing easier-to-use visualizations for code changes, as well as integrating with additional tools like the issue manager.

6.3 Tools for the build process

Build tools are highly developer-centered. Their goals relate to enabling *continuous delivery* and the *delivery* category of concepts. We have considered not only tools that generate deployable packages, but also called *builds*, but also tools that generate essential artifacts and feedback using the source code as input.

Each programming language has some tools to cope with the build process by supporting dependencies resolution, implementation of custom tasks, or generation of deployable packages. Examples of dependency managers, also called package managers, are Pip for Python, RubyGems for Ruby, NuGet for .NET, and Maven and Gradle that compete in the Java landscape. Gradle eases the implementation of custom tasks during the build, as also done by GNU Make, that is often used for GNU/Linux packages, or Rake for Ruby. Some of the cited tools, such as Maven, are also responsible for producing the deployable package, such as WAR files for Java environments. However, for some languages, like Python and Ruby, there is no need for producing a deployable package as a single-file artifact. It is also possible to use more generic package tools coupled to the target environment, such as Debian packages.

Each programming language also has some unit-test frameworks, which provide vital feedback for developers about the correctness of the software. Some examples are JUnit and Mockito for Java, RSpec for Ruby, and NUnit for .NET. More sophisticated testing which automates the end-user behavior for web applications is possible with browsing automation tools, such as Selenium.

Another type of feedback for developers is regarding source code quality, which is provided by source-code static analysis tools, such as SonarQube and Analizo [119]. SonarQube classifies code problems and evaluates coverage metrics as well as technical debts in several programming languages via its plugins. Analizo supports the extraction of a variety of source code metrics for C, C++, C#, and Java codes. In general, source-code analysis tools can point to issues in source-code, like non-compliance to the standard style, problems in maintainability, risk of bugs, and even vulnerability breaches. Source-code static analysis tools vary in supported programming languages and in the way they are delivered. They can be provided as-a-service, for example, Code Climate, or even within the developer environment through tools such as PMD for the Eclipse IDE.

One critical requirement for a build tool in DevOps context is automation. There are some products whose build actions can be only triggered through a Graphical User Interface (GUI), which is not acceptable in a continuous delivery process.

6.4 Tools for continuous integration

Continuous integration tools orchestrate several automated actions that, together, implement the *deployment pipeline* pattern. Among the stages orchestrated by the pipeline are: package generation, automated test execution for correctness verification, and deployment to both development and production environments. These tools are related to the *delivery* category of concepts.

The main actors responsible for defining the pipeline structure are typically developers. Operators usually collaborate on defining the deployment stages; they are also in charge of maintaining the continuous integration infrastructure running as a service to developers. For this, operators run continuous integration tools such as Jenkins or GitLab CI (about.gitlab.com/features/gitlab-ci-cd). Deployable packages can be stored on repositories like Nexus for enabling future rollback.

Since downtime of continuous-integration infrastructure results in the interruption of continuous delivery, it is common to use highly-available third-party services, such as GitLab.com and Travis.

6.5 Tools for deployment automation

Deployment automation tools are employed in the deployment stages of the pipeline to make the *continuous delivery* process viable. They enable frequent and reliable deployment processes, as well as other concepts related to the *delivery* category. Although the use of deployment automation tools is a joint effort between developers and operators, the primary mission of continuous delivery is putting the deployment schedule under business control.

Every automated deployment approach relies on the concept of “infrastructure as a code.” It requires engineers to specify servers, networking, and other infrastructure elements in files modeled after software code [102]. In this model, deployment and infrastructure definitions are shared among developers and operators, allowing effective cooperation between them.

Automated deployment can encompass not only the deployment of the package to the production environment but also the provisioning of the target environment. Such provisioning is usually performed in cloud environments [11], such as Amazon Web Services, Google Cloud, and Azure. These platforms deliver a vast amount of infrastructure services via Infrastructure as a Service (IaaS) model, like launching virtual machines, databases, queues, etc. The creation of all these resources can also be orchestrated, in Amazon’s platform, through the usage of AWS Cloud Formation. Operators and developers can share the task of using IaaS services.

It is also possible to use a Platform as a Service (PaaS), such as Heroku. In the PaaS approach, the platform is responsible for the deployment, and developers do not know the underlying virtual infrastructure. Serverless services – a new trend in cloud computing – aim to abstract servers for developers, which is close to PaaS; the main differences are the scaling transparency and the charge for computing usage [107]. These models suggest there is no need for specialized operations teams [11]; or, at least, that fewer people are required for operations [107].

If using IaaS, the environment provisioning is followed by the execution of scripts that effectively install the application in the target environment. A deployment script can be written using Shell Script, but configuration management tools such as Chef and Puppet offer advantages by leveraging operation system portability and idempotence mechanisms, which are difficult to achieve with Shell Script. An example of language construct of Chef that leads to portability is using the “package” resource, which is resolved to a concrete package manager, such as Apt, only at execution time. An example of an idempotent mechanism of Puppet in the “service” resource is declaring the desired final state of the service as “running” rather than writing a command to start the service [111].

Another alternative for deployment is containerization, primarily implemented by Docker. Docker containers resemble virtual machines. The main difference is that they are more lightweight [26], considering they share the kernel of the host. Docker and related tools, such as Docker Compose, Kubernetes, and Rancher, allow the specification of containers and their dependency relationships. These specifications generate container images in the build stage, which are later instantiated in the target environment. Docker has been used not only for deploying applications but also for deploying the underlying infrastructure [26].

Containerization could be seen as complementary to the deployment script strategy (done by Chef or Puppet). However, in practice, these strategies seem to compete. A Chef script is executed continuously on the target node, and its success depends on the previous target node state. On the other hand, in containerization the whole containerized environment is generated in the build, so the environment is destroyed and rebuilt at each new software version. When compared to the Chef strategy, Docker yields faster and more reliable deployment [26], but at the expense of bigger builds.

Complimentary usage of configuration management and containers entails the setup of Docker environment and update of the operating system software, like SSH. Another usage is to manage

configuration that varies across diverse environments (testing, staging, production). Such a configuration cannot be embedded in container images since the same image must be deployed in every environment. Nonetheless, the debate about configuration management versus containers is not fully explored in the literature. Among practitioners, although it is possible to find support for a complementary usage of both [54], there is a trend to favor containers over configuration management tools [69, 80, 86].

Zhu *et al.* compare the provisioning reliability of two approaches: using “lightly baked images” versus “heavily baked images” [50]. In the lightly baked approach, a virtual machine image is instantiated, and Chef is executed to install the application, whereas in the heavily baked approach the image already contains the entire application. This heavily baked approach is similar to the container strategy, but using virtual machine technology. The authors conclude that deployment using lightly baked images is less reliable since it involves more external resources during deployment, which tends to increase errors and delays.

As the application evolves, the database structure evolves as well. Traditionally, “database administrators” maintain the database structure. The adoption of emergent design [97] and the need for frequent releases have encouraged the use of database migrations tools such as Flyway, which controls the automated application of schema updates across different environments, thus enabling developers to manage database structure themselves. In this context, one must rethink the traditional role of database administrators.

Developers and operators share these tools for deployment. The main risk is the absence of clarity of responsibilities, which can cause friction in organizations during DevOps adoption [34]. Developers believe they do not control their application and feel they are doing someone else’s job. Additionally, operators may judge developers as unable to perform some tasks or operate some tools.

6.6 Tools for monitoring

Monitoring tools usually track applications’ non-functional properties, such as performance, availability, scalability, resilience, and *reliability*. Self healing, alerts, log management, and business metric follow up are example tasks performed by monitoring tools; they relate to the *runtime* category of concepts.

Example of tools for monitoring and alerting are Nagios, Zabbix, and Prometheus. Examples of log management tools are Graylog and Logstash. Cloud services also play an essential role in guarantying non-functional properties of applications since they provide elastic resources that can be allocated on demand [99]. It is also typical for cloud services to provide monitoring and alerting services.

The tendency toward cross-functional, full stack teams [2, 17], combined with the expectation that developers must be accountable for the product [18, 20] pushes the use of these tools to the development team. Therefore, once again, the responsibility for using and mastering specific tools is unclear.

6.7 Actors

The still undefined division of responsibilities in the DevOps world makes it difficult to associate a role to each DevOps tool. At the same time, the variety of DevOps tools seems to challenge the idea of a single person holding the title of a “DevOps engineer.” Even a whole cross-functional team may struggle to know all these tools. On the other hand, companies preserving the operations department may have difficulty in precisely defining which roles should use which tools.

Humble and Molesky suggest operations teams should provide IaaS to product teams [22], such as continuous integration platform, compute provisioning, and monitoring services. Traditional

Table 4. Major DevOps tools related to actors and DevOps concepts

Category	Examples	Actors	Goals	Concepts
Knowledge sharing	Rocket Chat GitLab wiki Redmine Trello	Everyone	Human collaboration	Culture of collaboration Sharing knowledge Breaking down silos Collaborate across departments
Source code management	Git SVN CVS ClearCase	Dev / Ops	Human collaboration Continuous delivery	Versioning Culture of collaboration Sharing knowledge Breaking down silos Collaborate across departments
Build process	Maven Gradle Rake JUnit Sonar	Dev	Continuous delivery	Release engineering Continuous delivery Automation Testing automation, Correctness Static analysis
Continuous Integration	Jenkins GitLab CI Travis Nexus	Dev / Ops	Continuous delivery	Frequent and reliable release process Release engineering Continuous integration Deployment pipeline Continuous delivery, Automation Artifact management
Deployment automation	Chef, Puppet Docker Heroku Open Stack AWS Cloud Formation Rancher Flyway	Dev / Ops	Continuous delivery Reliability	Frequent and reliable release process Release engineering Configuration management Continuous delivery Infrastructure as code Virtualization, Containerization Cloud services, Automation
Monitoring & Logging	Nagios Zabbix Prometheus Logstash Graylog	Ops / Dev	Reliability	You built it, you run it After-hours support for Devs Continuous runtime monitoring Performance, Availability, Scalability Resilience, Reliability, Automation Metrics, Alerting, Experiments Log management, Security

product teams become responsible for using these tools, whereas operations teams are responsible for set-up and maintenance, ensuring their performance and availability, and consulting in their usage. In this scenario, the operations team becomes itself a product team [22].

This pattern nears that of adopting outsourced development infrastructures, such as GitHub or GitLab. However, outsourcing the infrastructure has additional benefits, like freeing the team to experiment with different tools [20], and not only the ones supported by the operations staff. Most of the time, reputable infrastructure providers have better availability than in-house solutions. We discuss further the possible DevOps adoption approaches and their consequences in Section 8.2.

7 IMPLICATIONS FOR ENGINEERS, MANAGERS, AND RESEARCHERS

DevOps principles, practices, and tools are changing the software industry. However, many industry practitioners, both engineers and managers, are still not aware of how their daily work can be

affected by such principles, practices, and tools. By surveying the DevOps literature, we found several implications of DevOps adoption for industry practitioners and for academic researchers. In this section, we lay out such implications as a practical guide to help professionals to adapt to the significant impacts of DevOps in their fields.

Our contribution is to discuss the DevOps implications for each perspective individually: engineers, managers, and researchers. In these implications, readers will find issues they will likely confront, some core concerns they should have, and potential solutions already adopted by the community. We also outline DevOps-related topics that the academic community could exploit in the future. Finally, presenting these implications raises relevant considerations, preparing our discussion of unresolved DevOps challenges in Section 8.

7.1 Implications for engineers

Based on the reviewed literature, we list DevOps implications affecting how engineers must architect systems, interact with their peers, and even how to adopt processes, such as incident handling.

Microservices: adopting microservices architecture is recommended in conjunction with adopting continuous delivery [2, 42]. Loosely coupled and well-encapsulated microservices architecture lead to good system testability and deployability [21]. Since microservices do not come without new challenges, we further discuss this topic in Section 8.1.

Cloud services: architecture patterns involving cloud services can assist in the deployment and operation of applications [11], diminishing the need for dedicated operations teams [107]. It is particularly useful in the context of cross-functional teams.

Rolling back: several authors state that DevOps engineers should be able to roll back applications in case of problems after deployment [6, 10]. However, in scenarios with complex integration or database evolution [6], rolling back can be a tricky task [87]. Alternatives such as feature toggle [79] or handling the root causes of problematic releases can avoid rolling back [61, 87].

Embedded systems and IoT devices: deployment on embedded systems can be hard, especially when only the customer owns and controls the hardware platform [31]. Therefore, engineers must customize the updating mechanisms for embedded systems and IoT devices.

Inhibitors for high-frequency delivery: engineers must be aware of scenarios that could impose barriers to high-frequency delivery. Updating the software of automation control systems can require a factory to stop its production [29], which can be costly. Traditional techniques to avoid downtime on deployment, such as Blue Green Deployment [74] and Canary Release [110], may not be suitable for critical applications, such as for factories, and medical equipment. Another less critical scenario, such as mobile apps, can wait for a release cycle to publish a new update [29].

Testing: although companies recognize the importance of automated testing, they still struggle to implement it [35] fully. It is especially true for user interface tests automation [29]. Other factors that make automated testing complex are hardware availability for load testing and user experiment assessment [29]. Parallelizing test suites can be necessary to reduce testing time [33]. Tests that seem to fail at random called “flaky tests,” are not acceptable [33].

Quality assurance team: quality assurance skills are necessary for locating specific error scenarios and corner cases. However, preserving the quality assurance team separate from the development team is questionable. DevOps and agile practices require a change in the role of quality assurance teams, or even its elimination [4].

Legacy systems: although it takes a great deal of work to achieve continuous delivery on main-frame platforms, there are successful reports of it [21]. Some legacy architectures might not be designed to run automated tests [29]. Nonetheless, teams must be aware that cultural factors, such as managers who say “This is the way we have always done it” [21], can limit the adoption of continuous delivery more than technical factors.

Communication: improving communication among organizational silos is key to DevOps adoption [22]. However, practitioners must be aware that effective interdepartmental communication is still a challenge, especially within remote teams [15].

Learning: software professionals must be prepared to learn new tools, focusing on automation [63]. They must also cope with the ever-changing nature of these tools, which implies maintaining heterogeneous environments and migrating technologies [42].

Building the deployment pipeline: the benefits delivered by a deployment pipeline are many. However, engineers must be aware that setting up the infrastructure for continuous deployment can demand a considerable effort [29, 33]. Breaking down the system into microservices also requires building multiple pipelines. Engineers should not try to build all the continuous delivery ecosystem in a single step: an approach based on continuous improvement is preferable [33].

Pipeline maintenance: pipeline execution generates a lot of artifacts, such as build and logs. Artifacts such as production logs, bug tracks, parameters configuration, and temporary files, must be appropriately archived and removed at some point [36]. Despite its importance to pipeline sustainability, organizations often overlook this maintenance process [36].

On-boarding: the automation built for carrying continuous delivery also promotes faster on-boarding of new members to the team [33].

Incident handling: software developers must be educated in software security and must cope with incident handling [24], bug tracking, systems failure, or take over primary responsibility for this activity.

Coding for stability and security: although software stability and security are traditional operational concerns, in a DevOps context such non-functional requirements must be leveraged by software implementation, including the code for deployment automation and a consideration of the possibility of failures and delays in the underlying infrastructure [30, 38, 50].

7.2 Implications for managers

Based on the reviewed literature, we list implications related to how managers must face the DevOps phenomenon: required management and cultural paradigms, training people, structuring and assessing the DevOps-adoption process, as well the expected outcomes from this process.

Adoption of lean principles: since DevOps is based on lean principles [10], organizations eager for DevOps adoption should take a step back and learn them [105]. In particular, Kim recommends [89]: 1) mapping the value stream for optimizing global system performance, rather than for local optimization; 2) amplifying continuous feedback loops to support necessary corrections; and 3) improving daily work through a culture promoting frequent experimentation, risk-taking, learning from mistakes, and knowing that practice and repetition are prerequisites to mastery.

DevOps adoption: how to deploy DevOps in an organization is a critical question for managers. However, the lack of a consensual definition of DevOps is a reason for making it a hard decision. Many studies explore this subject, organizations still struggle with it, and we discuss it in more depth in Section 8.2.

Assessment: an organization should be able to measure the success of a DevOps adoption process. Nonetheless, any top-down imposition of a metric-based evaluation must be used with care. If personal evaluation depends on such metrics, engineers can focus on producing good numbers rather than improving the software process [61]. We further discuss how to assess the quality of DevOps practices in Section 8.3.

Training: DevOps demands additional technical skills from software professionals. Developers must acquire skills from operators and vice versa. It is necessary to conduct training in using not only tools but also in using DevOps concepts to keep pace with tooling's rapid pace evolution [42]. Top management sponsorship for training is often necessary for many contexts.

Job titles: Defining job titles impacts hiring and training. Although the DevOps movement emerged to approximate developers and operators, nowadays, the industry adopts the role of DevOps engineer, which executes tasks mostly linked to scripting automation and CI/CD practices [23]. However, this role blurs with other ones, such as release engineer and build engineer. There is no consensus in industry and academia when defining these roles. By analyzing hiring ads, Hussain *et al.* found that DevOps positions usually do not impose build and release management as attributions [23], whereas Kerzazi and Adams found that these three roles share common activities [88]. The challenge for managers to define job titles is also related to how to structure development and operations teams, which we discuss in Section 8.2.

Culture: Humble stated that, typically, the obstacle to continuous delivery adoption is not the skill level of individual employees, but failures at the management and leadership level [21]. High-performing organizations strive, at all levels, towards continuous improvement, rather than treating workers as fungible "resources" that should merely execute tasks as efficiently as possible [21]. Top and low-level management are responsible for creating an environment in which failure is allowed, and people seek continuous improvement.

Increase in delivery throughput: Neely and Stolt reported that the delivery throughput per developer in their company increased with the adoption of continuous delivery and defect rate became more predictable [33]. Siqueira *et al.* also reported how, after investments in DevOps and continuous delivery, the number of releases per semester remained the same even after a considerable decrease in the number of team members [44].

Building trust: when an organization builds software for other large organization, especially the government, it is common for the relationship between the contractor and contracted to be dominated by mistrust, which leads to cumbersome development processes. However, Siqueira *et al.* advocate, based on their experience, that continuous delivery leverages a trust relationship among the contractor and contracted in software development, even in governmental projects [44].

Building for the government: although building software for government involves more bureaucratic processes, requirements and prioritization can often change due to political reasons [44], which leads to the need to shorten the release cycle by adopting agile and DevOps practices in the governmental scenario.

7.3 Implications for researchers

Based on the reviewed literature, we list DevOps-related open topics that the academic community could exploit in future research.

Software Architecture: some authors explore software design in the context of DevOps, continuous delivery, and continuous deployment [5, 42, 48]. However, engineers may still struggle with this in practice, since achieving the desired architecture can be infeasible in a single first DevOps

project. Researchers should investigate transitioning strategies to adopt practices and architectural changes.

Education: DevOps adoption requires more skilled software engineers. A broader investigation is needed to explore how to teach operations skills to developers and vice-versa, as well as to introduce software operation topics in software engineering courses [8, 9]. We discuss this in more detail in Section 8.4.

Embedded systems and IoT: continuously delivering new software versions in embedded systems is still a hard question [31, 35]. How to effectively adopt DevOps in an IoT context is an open research question to be investigated.

Compliance: DevOps provides both benefits and obstacles to regulatory compliance [21, 22, 28]. Research should be done to detect scenarios in which a high-frequency delivery is indeed not welcome or how one can demonstrate that automated deployment can help organizations comply with regulations, as stated by Humble [21]. Moreover, some practitioners advocate that engineers must have unrestricted access to production data [52, 85], which can be controversial in specific environments, such as financial systems.

Security: high-frequency of continuous delivery poses questions about the security in a DevOps context [25, 49]. However, DevOps can also bring benefits for security. This dual relationship generates a dichotomy that should be further investigated.

Testing large-scale distributed systems: some errors in large-scale distributed systems can be hard to reproduce in testing environments [20]. The chaos engineering approach advocates running experiments in production [3]. This is an incipient topic in the research literature.

Quantitative assessment metrics: few works use adequate quantitative metrics for assessing DevOps as is the case of the Puppet State of DevOps Reports [121]. The investigation and use of such quantitative metrics should be intensified in related research.

Deployment approach: two automated deployment approaches are the container-based one [26] and the deployment scripts written in domain-specific languages such as Chef. Investigating how complementary or exclusive these approaches are can help engineers choose the best toolset.

Improving interdepartmental communication: how to effectively enable interdepartmental communication is a challenge, especially for distributed organizations [15]. Better communication does not merely mean more communication since too much communication, and an excess of meetings can negatively impact productivity.

Adoption strategies: there are still many open questions about how organizations should adopt DevOps. It is stated that DevOps adoption requires top-management support [10, 35]. Sometimes it does not happen in the first moment, and a “guerrilla” strategy can take place (i.e., acting outside the company standard procedures). Moreover, arguments to encourage DevOps adoption can differ from engineers to managers [33]. In such context, researchers can support DevOps adoption by providing a further investigation into balancing top-management support and guerrilla strategies; studying arguments to convince managers; providing guidelines about which DevOps-adoption strategy to choose based on organizational characteristics.

Investigation involving multiple organizations: since most empirical research is conducted with data from a single organization [10, 15, 34, 34], any conclusion from these works deserves further investigation. Surveying more companies, as done by Leppanen *et al.* [29], can strengthen or weaken previous findings.

Other research topics: Claps *et al.* list other technical and social challenges for continuous deployment adoption, such as product marketing, team coordination, customer adoption, feature discovery, plugin management, cross-product dependencies, and scaling CI tools [10]. Olsson *et al.* list among the challenges in continuous delivery adoption: coordination of supplier integration, business models, and difficulty in overviewing projects status [35].

8 UNRESOLVED CHALLENGES

Throughout this survey, we have shown that overviewing DevOps concepts and tools raises some issues to debate. The previous section also listed some not-so-straightforward implications for practitioners and researchers. Therefore, based on the surveyed literature and additional sources of knowledge, we further discuss in this section some of the DevOps challenges faced by managers, engineers, and researchers that are not thoroughly handled by the current state-of-the-art.

8.1 How to re-design systems toward continuous delivery

Highly coupled monolithic architectures are obstacles to effective continuous delivery. Complex dependency management of software components and teams is imposed to the deployment pipeline [42]. An essential principle for successfully adopting and implementing continuous delivery is an architecture composed of small and independently deployable units, also called *microservices* [42]. In such architectural style, services interact through the network and are built around business capabilities [95].

Conceiving of an application architecture with loosely coupled and well-encapsulated microservices guarantees two architectural attributes required by continuous delivery: *testability* and *deployability* [21]. If each microservice has its test suite, this reduces the blocking of deployment due to long-running tests [29].

Whereas some authors say microservices facilitate effective implementation of DevOps [2], others say microservices require DevOps [17], since deployment automation minimizes the overhead to manage a significant number of microservices. However, adopting microservices comes with several challenges. First, there is heterogeneity in non-functional patterns such as “startup scripts, configuration files, administration endpoints, and logging locations” [6]. Technological heterogeneity can be a productivity barrier for newcomers in the team. Second, microservices must be deployed to production with the same set of versions used for integration tests [5]. These challenges can be overcome by adopting a minimum set of microservices standards across the organization [6].

Common microservice management patterns associated with DevOps are: one deployment pipeline per microservice; log aggregator; service registry; correlation ID’s [5]; and segregation of source code, configuration, and environment specification [2]. Other complimentary patterns are strangler application [73][5, 21], load balancer, consumer-driven contracts [2], circuit breaker [104], backwards compatibility, and versioned APIs [21]. However, the last one is controversial, and some communities usually do not recommend microservice versioning [2].

In highly regulated environments, the proper use of microservices can constrain unfriendly regulations to specific system modules and people [21]. Additionally, the usage of a platform-as-a-service (PaaS) can automate much of the compliance checking, a model used, for example, by the U.S. federal government [21]. PaaS is also recommended for its operational simplicity, like other cloud services, such as storage services, asynchronous processing with queues, email delivery, and real-time user monitoring [11].

Finally, although reusability is a historical goal of software engineering [98], Shahin *et al.* recommend engineers not to focus too much on it [42]. It brings coupling, which is a huge bottleneck

to continuous delivery [42]. Therefore, each team should discuss the trade-off between reusability and independence from other components, services, and teams.

8.2 How to deploy DevOps in an organization

A hard question not yet fully answered by the literature is how – perhaps whether – an organization should be restructured to adopt DevOps. We believe the literature is incomplete and even contradictory regarding this subject. We discuss a few studies that have evaluated this matter.

The seminal paper of Humble and Molesky about DevOps presents three scenarios. First, preserving the structures of development and operations departments, DevOps is led by human collaboration among developers and operators, with operators attending agile ceremonies and developers contributing to incident solving [22, 24]. Alternatively, product teams, also called cross-functional teams, effectively incorporate operators. Finally, in a third scenario, one product team is composed of only operators offering support services (continuous integration, monitoring services) to the entire organization.

Similarly, Nybom *et al.* also present three distinct scenarios to DevOps adoption [34]: *i*) collaboration among development and operations departments; *ii*) cross-functional teams; and *iii*) creation of “DevOps teams.” These approaches are summarized in the Table 5 and discussed in detail in the following paragraphs.

Table 5. DevOps adoption approaches

Collaborating departments
Development and operations departments collaborate closely. It implies overlapping of developers and operators responsibilities. <i>Downside:</i> new responsibilities can be unclear for employees.
Cross-functional team
The product team is responsible for deploying and operating (<i>You built it, you run it</i>). Recommended by Amazon and Facebook. <i>Downside:</i> requires more skilled engineers.
DevOps team
Acts as a bridge between developers and operators. It is better accepted when it is a temporary strategy for cultural transformation. <i>Downside:</i> risk of creating a third silo.

In the first approach, the responsibilities of developers overlap with operators [10, 15, 34, 42]. High risk for adopting this strategy occurs when new responsibilities do not become evident in the organization [10, 34], which could lead to frictions among developers and operators [7]. This is especially true when delivery automation is not prioritized [34].

Cross-functional teams resemble the “whole team” practice from XP [57], and seems to prevail in literature [2, 11, 18, 20]. This structure appeals to “T-shaped” professionals, who have expertise in few fields and basic skills in multiple correlated areas [14][76]. In this model, at least one team member must master operations skills. From this perspective, the so-called “DevOps engineer” [23], also called the “full stack engineer” [23], is known as a developer with operations skills.

One could wonder whether it makes sense to talk about developers and operators collaboration in cross-functional teams context. Adopting cross-functional teams may just be a matter of moving operators to product teams. However, this is not trivial, considering that large organizations usually have only a small pool of operators for several development teams [34]. Therefore, companies must hire people with both development and operations skills or train some of their developers in operations skills. Nevertheless, pressuring developers to learn operations can be ineffective, as

they are already overwhelmed with other skills they need to learn. Transforming development teams into product teams has still another major cultural shift, as developers become responsible for 24/7 service support [14, 42]. Even when services should be designed to remain available without human interaction [78]. An extra challenge for cross-functional teams is guaranteeing that specialists interact with their peer groups to share novelties in the field [14]. Spotify, for example, achieves this through inter-team “guilds,” which are communities of common interest within the organization [92].

Assigning a “DevOps team” as a bridge between developers and operators [34] has become a trend [121]. However, it is not clear what precisely “DevOps teams” are, and how they differ from regular operations teams [34][121]. This approach is criticized by Humble, who considers that potentially creating a new silo is not the best policy to handle the DevOps principle of breaking down walls between silos [82]. Skelton and Pais present some variations, called “DevOps topologies,” and also DevOps anti-patterns [114]. They argue that, although the DevOps team silo is an anti-pattern, it is acceptable when it is temporary and focused on sharing development practices to operations staff and operations concerns to developers. Siqueira *et al.* report how a DevOps team with senior developers and rotating roles among members was a key decision to construct and maintain a continuous delivery process [44].

There is yet the strategy adopted by Google, called Site Reliability Engineering (SRE) [58], which involves the evolution of the operations engineer role. In addition to product teams, Google has SRE teams responsible for product reliability engineering, what includes performance, availability, monitoring, and emergency response. SRE teams have an upper limit of using 50 percent of their time on operational tasks because they must allocate at least 50 percent of their time increasing product reliability through engineering and development activities.

One should also question the impact of deployment automation on operators’ roles. Chen reported that, after continuous delivery adoption, operations engineers only needed to click a button to release new versions [7]. One should question the role of an engineer in such a context. Moreover, if software updates are adequately broken down into small increments, and delivery automation turns deployment in a “non-event” [22], practices such as “development and operations teams should celebrate successful releases together” [22] can be perceived as contradictory.

Therefore, if operations provide support services (continuous integration, monitoring) [22] and product reliability is appropriately designed, product teams can easily deploy and operate their services. Thus, one can even replace operations with third-party cloud services, as done by Cukier [11], and achieve an organizational structure known as “NoOps”. Some blog posts promote NoOps concepts, but the literature has not yet covered this subject. One possible definition is: “NoOps (no operations) is the concept that an IT environment can become so automated and abstracted from the underlying infrastructure that there is no need for a dedicated team to manage software in-house” [109]. Cross-functional teams can embrace NoOps, with a particular focus on employing cloud technology that automates much of the operational work, rather than reallocating operators or teaching developers all the operational work.

8.3 How to assess the quality of DevOps practices in organizations

Some researchers have proposed maturity models for DevOps adoption and continuous delivery adoption [29, 51, 56]. Feijter *et al.*, for example, provide maturity DevOps models to help organizations measure their current maturity level and to determine areas that require additional investment [13]. However, none of these maturity models are widely adopted by industry. Indeed, most organizations have almost no visibility or reliable measurement of their software-delivery practices [19].

Forsgren and Kersten claim that both survey data and system data [19] should measure DevOps transformations. System data can provide a continuous flow of information, although setting up the aggregation of metrics from different sources can be challenging. Survey data provides a holistic view of out-of-the system issues, such as culture, job satisfaction, and burnout, and can even point to problems on system-data collection.

Whether survey or system data, Forsgren and Kersten still make clear that if results are used to punish teams, data collection will be unreliable [19], as also reinforced by Brown from Microsoft [61]. Kua cautions that inappropriate use of metrics can lead to undesired behavior and even divert the organization from its goals [94]. Kua also compiles guidelines for better usage of metrics by: explicitly linking metrics to goals, favoring trends over absolute numbers, tracking shorter periods, and changing metrics when they do not drive change anymore.

Some examples of metrics based on survey data used on the State of DevOps Reports [53, 121] are: IT performance as a function of time from commit to deployment, frequency of deployment, and recovery time. Time spent on unplanned work and rework. Typology of organizational culture (pathological, bureaucratic or generative) based on climate for learning, job satisfaction, developers and operators having win-win relationships, usage of version control, automated testing. Employee engagement, based on the Net Promoter Score, indicating how likely employees are to recommend the company products and services.

Snyder and Curtis use metrics to evaluate DevOps in a specific company [45]. Metrics such as productivity, defect ratio, dollars spent, cycle time release, and build count were aggregated in a “total quality index”. More importantly, the aggregation of data from different silos was used to align organizational efforts. The authors observed a higher increase in the total quality index of agile and DevOps teams, which helped executives justify investments in the agile-DevOps transformation.

There is a severe lack of industry consensus on how to measure software delivery [19], and designing a good survey requires some expertise. Thus, researchers should provide standards for collecting and analyzing evaluation metrics for the software delivery process, as done by Feijter *et al.* [13]. In the health sector, this model is mature, wherein researchers instrument therapists with standard protocols for diagnoses purposes. Nonetheless, Forsgren and Kersten provide some general guidelines for survey application: it should be limited to 20 to 25 minutes and applied every four to six months [19]. Forsgren and Kersten also advise avoiding surveying management and executives, since they tend to overestimate the maturity of their organizations.

Researchers can also run the surveys across multiple organizations as done in the State of DevOps Reports. The results of these questionnaires can help practitioners in orienting their careers and help organizations to self-evaluate their performance by comparing themselves to their peers.

8.4 How to qualify engineers for DevOps practice

More than one-hundred DevOps tools are available [63], and they are continuously evolving [42]. Professionals must be able to choose which tools to study in depth, which tools to have some familiarity with, which tools to test, and which ones to ignore. Such choices must be based on a conceptual analysis of personal and organizational demands and perspectives. Our conceptual maps (Section 5) and our table of major DevOps tools (Table 4) may guide engineers to prioritize the improvement of their professional skills.

Care is needed when managers assume that employees lack competence. When questioned “Where do you get Netflix’s amazing employees from?”, a Netflix architect replied: “I get them from you!” [21]. The lesson here is that a culture of autonomy and responsibility [18, 20] is an important motivating factor for employees continuous learning.

In software engineering courses, operations skills, often neglected in college education, become mandatory. However, elaborate effective DevOps programs are challenging. Exercises that demand

students build distributed systems with adequate availability, scalability, and performance require not only some infrastructure, but also efficient correction of such exercises with automated checks in submissions [8]. Although there are stable platforms for auto-grading coding assignments (autolab.github.io) [101], evaluating infrastructure code and non-functional concerns are harder to automate. Christensen used Docker in the classroom to develop a small set of scripts to automate some tasks in executing and assessing submissions [8]. However, he did not achieve automation of exercise correction due to the nontrivial task of setting up a multi-server environment to detect errors in students' submissions. Bai *et al.* automate the generation of assessment reports based on patterns of tasks, commits, branches, tests, and the source code itself [1].

Knowledge in software engineering is always evolving, and any software engineer must also have self-learning skills to overcome daily basis challenges. Social interactions on Internet forums are a vital source of information in the daily activities of engineers. Believing that it can be enhanced, Magoutis *et al.* built a social network that helps DevOps engineers analyze results of past deployment executions as well as communicate and exchange ideas to better understand trade-offs in the space of deployment solutions [32]. The work of Magoutis *et al.* show evidence that more can be done to support engineers' self-learning in their daily activities.

9 LIMITATIONS OF THIS STUDY

Given the vast amount of work on DevOps, it was not feasible to conduct an exhaustive search in all available sources. We have not considered academic workshops, and we mentioned only a few books in the field. It would not be practical to try to cover every existing work on DevOps and to condense it all in this single survey. However, by focusing on journal and conference papers, and by applying the snowballing process, we aimed to cover the primary literature in the field. We decided not to expand the query string to avoid artificially favoring some of the DevOps concepts over others. However, the appearance of the keyword "DevOps" was not mandatory in the snowballing process so that we could select vital work tackling highly related subjects such as continuous delivery.

The choice of core and relevant papers may suffer from subjective bias. To reduce this bias, at least two authors conducted the selection process and the classification of papers. Senior researchers also oversaw the whole process. Our study also suffers the publication bias, which is the propensity of researchers and practitioners to report (and for referees to accept) more about positive results than negative results. We did not find, for example, a paper reporting a case of failed DevOps adoption. We did not address this issue in this study.

10 CONCLUSIONS

In this survey, we have discussed DevOps concepts and challenges presented in the literature. By associating these concepts with tools, we contributed to supporting practitioners in choosing a proper toolset. This paper also aides IT professionals by presenting in a systematic way the most relevant concepts, tools, and implications, associated with the professional perspectives of researchers, managers, and engineers – including developers and operators. We hope our reader can now better understand the impact of DevOps on daily activities based on each profile.

Two pillars of DevOps are 1) human collaboration across departments and 2) automation. We found that technical issues regarding delivery automation are more consolidated, with only a few minor controversies in the literature. On the other hand, there is no consensus on how to effectively empower collaboration among departments, and there are only a few tools for tackling this issue.

A technical topic highly related to DevOps revealed by our sources was microservice architecture. Some technical conflicting points are: there is no consensus on whether microservices should be versioned; some practitioners see configuration management tools and containerization as

complementary approaches, but others understand them as competitors; maximizing reusability of software components and services may not be the better strategy; rolling back is usually seen as a DevOps practice, but some practitioners tend to favor feature toggles to avoid these rollbacks.

Structuring an organization to introduce DevOps is now a major concern. There are three alternative structures: 1) preserving collaborating departments, 2) building cross-functional teams, and 3) having DevOps teams. Each one of these strategies has its challenges and, by following the current literature, it is not possible to define the best strategy for a given scenario. Therefore, exploring these and other structures in real organizations and how these organizations handle the “DevOps role” are promising topics for future research. However, whatever the structure an organization adopts, it is clear that the DevOps movement has irreversibly blurred the frontier between developers and operators, even for organizations that have not yet fully embraced DevOps.

ACKNOWLEDGEMENTS

We thank the Brazilian Ministry of Citizenship, via the TAIS project, and the Brazilian Federal Data Processing Service (Serpro) for the support. This research is also part of the INCT of the Future Internet funded by CNPq proc. 465446/2014-0, Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001, FAPESP proc. 14/50937-1, and FAPESP proc. 15/24485-9.

LIST OF CORE PAPERS

- [1] X. Bai, M. Li, D. Pei, S. Li, and D. Ye. 2018. Continuous Delivery of Personalized Assessment and Feedback in Agile Software Engineering Projects. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET '18)*. 58–67. Code: I818.
- [2] Armin Balalaie, Abbas Heydarnoori, and Pooyan Jamshidi. 2016. Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture. *IEEE Software* 33, 3 (2016), 42–52. Code: A2.
- [3] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal. 2016. Chaos Engineering. *IEEE Software* 33, 3 (2016), 35–41. Code: A76.
- [4] Len Bass. 2018. The Software Architect and DevOps. *IEEE Software* 35, 1 (2018), 8–10. Code: I33.
- [5] Kyle Brown and Bobby Woolf. 2016. Implementation Patterns for Microservices Architectures. In *Proceedings of the 23rd Conference on Pattern Languages of Programs (PLoP '16)*. The Hillside Group, Article 7, 7:1–7:35 pages. Code: A104.
- [6] Matt Callanan and Alexandra Spillane. 2016. DevOps: Making It Easy to Do the Right Thing. *IEEE Software* 33, 3 (2016), 53–59. Code: A67.
- [7] Lianping Chen. 2015. Continuous delivery: Huge benefits, but challenges too. *IEEE Software* 32, 2 (2015), 50–54. Code: B15.
- [8] Henrik Bærbak Christensen. 2016. Teaching DevOps and Cloud Computing Using a Cognitive Apprenticeship and Story-Telling Approach. In *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education (ITICSE '16)*. ACM, 174–179. Code: A47.
- [9] Sam Chung and Soon Bang. 2016. Identifying Knowledge, Skills, and Abilities (KSA) for Devops-aware Server Side Web Application with the Grounded Theory. *J. Comput. Sci. Coll.* 32, 1 (2016), 110–116. Code: A16.
- [10] Gerry Gerard Claps, Richard Berntsson Svensson, and Aybüke Aurum. 2015. On the journey to continuous deployment: Technical and social challenges along the way. *Information and Software Technology* 57 (2015), 21–31. Code: B13.
- [11] Daniel Cukier. 2013. DevOps Patterns to Scale Web Applications Using Cloud Services. In *Proceedings of the 2013 Companion Publication for Conference on Systems, Programming, & Applications: Software for Humanity (SPLASH '13)*. ACM, 143–152. Code: A35.
- [12] Maximilien de Bayser, Leonardo G. Azevedo, and Renato Cerqueira. 2015. ResearchOps: The case for DevOps in scientific applications. In *2015 IFIP/IEEE International Symposium on Integrated Network Management (IM)*. 1398–1404. Code: I40.
- [13] Rico de Feijter, Sietse Overbeek, Rob van Vliet, Erik Jagroep, and Sjaak Brinkkemper. 2018. DevOps Competences and Maturity for Software Producing Organizations. In *Enterprise, Business-Process and Information Systems Modeling*. Springer, 244–259. Code: S805.
- [14] Patrick Debois. 2011. Devops: A software revolution in the making. *Cutter IT Journal* 24, 8 (2011), 3–5. Code: B4.
- [15] Elisa Diel, Sabrina Marczak, and Daniela S. Cruzes. 2016. Communication Challenges and Strategies in Distributed DevOps. In *11th IEEE International Conference on Global Software Engineering (ICGSE)*. 24–28. Code: I19.

- [16] Andrej Dyck, Ralf Penners, and Horst Lichter. 2015. Towards Definitions for Release Engineering and DevOps. In *2015 IEEE/ACM 3rd International Workshop on Release Engineering*. 3–3. Code: B26.
- [17] C. Ebert, G. Gallardo, J. Hernantes, and N. Serrano. 2016. DevOps. *IEEE Software* 33, 3 (2016), 94–100. Code: A54.
- [18] Dror G Feitelson, Eitan Frachtenberg, and Kent L Beck. 2013. Development and deployment at Facebook. *IEEE Internet Computing* 17, 4 (2013), 8–17. Code: B7.
- [19] Nicole Forsgren and Mik Kersten. 2018. DevOps Metrics. *Commun. ACM* 61, 4 (2018), 44–48. Code: B21.
- [20] Jim Gray. 2006. A conversation with Werner Vogels. *ACM Queue* 4, 4 (2006), 14–22. Code: B3.
- [21] Jez Humble. 2017. Continuous Delivery Sounds Great, but Will It Work Here? *Queue* 15, 6 (2017), 57–76. Code: B22.
- [22] Jez Humble and Joanne Molesky. 2011. Why enterprises must adopt DevOps to enable continuous delivery. *Cutter IT Journal* 24, 8 (2011), 6. Code: B5.
- [23] Waqar Hussain, Tony Clear, and Stephen MacDonell. 2017. Emerging Trends for Global DevOps: A New Zealand Perspective. In *Proceedings of the 12th International Conference on Global Software Engineering (ICGSE '17)*. IEEE Press, 21–30. Code: A25.
- [24] Martin Gilje Jaatun. 2018. Software Security Activities that Support Incident Management in Secure DevOps. In *Proceedings of the 13th International Conference on Availability, Reliability and Security (ARES 2018)*. ACM, 8:1–8:6. Code: A803.
- [25] Martin Gilje Jaatun, Daniela S. Cruzes, and Jesus Luna. 2017. DevOps for Better Software Security in the Cloud. In *Proceedings of the 12th International Conference on Availability, Reliability and Security (ARES '17)*. ACM, Article 69, 69:1–69:6 pages. Code: A85.
- [26] Hui Kang, Michael Le, and Shu Tao. 2016. Container and Microservice Driven Design for Cloud Infrastructure DevOps. In *2016 IEEE International Conference on Cloud Engineering (IC2E)*. 202–211. Code: I58.
- [27] Mik Kersten. 2018. A Cambrian Explosion of DevOps Tools. *IEEE Software* 35, 2 (2018), 14–17. Code: I808.
- [28] Teemu Laukkarinen, Kati Kuusinen, and Tommi Mikkonen. 2017. DevOps in Regulated Software Development: Case Medical Devices. In *Proceedings of the 39th International Conference on Software Engineering: New Ideas and Emerging Results Track (ICSE-NIER '17)*. IEEE Press, 15–18. Code: A26.
- [29] M. Leppanen, S. Makinen, M. Pagels, V. Eloranta, J. Itkonen, M. V. Mantyla, and T. Mannisto. 2015. The highways and country roads to continuous deployment. *IEEE Software* 32, 2 (2015), 64–72. Code: B14.
- [30] Z. Li, Q. Lu, L. Zhu, X. Xu, Y. Liu, and W. Zhang. 2018. An Empirical Study of Cloud API Issues. *IEEE Cloud Computing* 5, 2 (2018), 58–72. Code: I802.
- [31] Lucy Ellen Lwakatare, Teemu Karvonen, Tanja Sauvola, Pasi Kuvaja, Helena Holmström Olsson, Jan Bosch, and Markku Oivo. 2016. Towards DevOps in the embedded systems domain: Why is it so hard?. In *49th Hawaii International Conference on System Sciences (HICSS)*. IEEE, 5437–5446. Code: A42.
- [32] Kostas Magoutis, Christos Papoulas, Antonis Papaioannou, Flora Karniavoura, Dimitrios-Georgios Akestoridis, Nikos Parotsidis, Maria Korozi, Asterios Leonidis, Stavroula Ntoa, and Constantine Stephanidis. 2015. Design and implementation of a social networking platform for cloud deployment specialists. *Journal of Internet Services and Applications* 6, 1 (2015). Code: S3.
- [33] Steve Neely and Steve Stolt. 2013. Continuous Delivery? Easy! Just Change Everything (Well, Maybe It Is Not That Easy). In *2013 Agile Conference*. 121–128. Code: B23.
- [34] Kristian Nybom, Jens Smeds, and Ivan Porres. 2016. On the Impact of Mixing Responsibilities Between Devs and Ops. In *International Conference on Agile Software Development (XP 2016)*. Springer International Publishing, 131–143. Code: S18.
- [35] Helena H. Olsson, Hiva Alahyari, and Jan Bosch. 2012. Climbing the "Stairway to Heaven" – A Multiple-Case Study Exploring Barriers in the Transition from Agile Development towards Continuous Deployment of Software. In *38th Euromicro Conference on Software Engineering and Advanced Applications*. 392–399. Code: B17.
- [36] Candy Pang and Abram Hindle. 2016. Continuous Maintenance. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 458–462. Code: I55.
- [37] Rahul Punjabi and Ruhi Bajaj. 2016. User stories to user reality: A DevOps approach for the cloud. In *2016 IEEE International Conference on Recent Trends in Electronics, Information Communication Technology (RTEICT)*. 658–662. Code: I17.
- [38] Akond Rahman. 2018. Characteristics of Defective Infrastructure As Code Scripts in DevOps. In *Proceedings of the 40th International Conference on Software Engineering (ICSE '18)*. ACM, 476–479. Code: A806.
- [39] M. Rajkumar, A. K. Pole, V. S. Adige, and P. Mahanta. 2016. DevOps culture and its impact on cloud delivery and software development. In *2016 International Conference on Advances in Computing, Communication, Automation (ICACCA)*. 1–6. Code: I48.
- [40] James Roche. 2013. Adopting DevOps Practices in Quality Assurance. *Commun. ACM* 56, 11 (2013), 38–43. Code: A74.
- [41] S. Van Rossem, W. Tavernier, D. Colle, M. Pickavet, and P. Demeester. 2018. Introducing Development Features for Virtualized Network Services. *IEEE Communications Magazine* 56, 8 (2018), 184–192. Code: I77.

- [42] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2016. The Intersection of Continuous Deployment and Architecting Process: Practitioners' Perspectives. In *Proceedings of the 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM '16)*. ACM, 44:1–44:10. Code: A64.
- [43] Alan Sill. 2014. Cloud Standards and the Spectrum of Development. *IEEE Cloud Computing* 1, 3 (2014), 15–19. Code: I67.
- [44] Rodrigo Siqueira, Diego Camarinha, Melissa Wen, Paulo Meirelles, and Fabio Kon. 2018. Continuous Delivery: Building Trust in a Large-Scale, Complex Government Organization. *IEEE Software* 35, 2 (2018), 38–43. Code: B29.
- [45] Barry Snyder and Bill Curtis. 2018. Using Analytics to Guide Improvement During an Agile/DevOps Transformation. *IEEE Software* 35, 1 (2018), 78–83. Code: I7.
- [46] Johannes Wettinger, Vasilios Andrikopoulos, and Frank Leymann. 2015. Automated Capturing and Systematic Usage of DevOps Knowledge for Cloud Applications. In *2015 IEEE International Conference on Cloud Engineering*. IEEE, 60–65. Code: A61.
- [47] Johannes Wettinger, Vasilios Andrikopoulos, and Frank Leymann. 2015. Enabling DevOps Collaboration and Continuous Delivery Using Diverse Application Environments. In *On the Move to Meaningful Internet Systems (OTM 2015 Conferences)*. Springer International Publishing, 348–358. Code: A17.
- [48] Eoin Woods. 2016. Operational: The Forgotten Architectural View. *IEEE Software* 33, 3 (2016), 20–23. Code: A82.
- [49] Hasan Yasar and Kiriakos Kontostathis. 2016. Where to Integrate Security Practices on DevOps Platform. *International Journal of Secure Software Engineering (IJ SSE)* 7, 4 (2016), 39–50. Code: A58.
- [50] L. Zhu, D. Xu, A. B. Tran, X. Xu, L. Bass, I. Weber, and S. Dwarakanathan. 2015. Achieving Reliable High-Frequency Releases in Cloud Environments. *IEEE Software* 32, 2 (2015), 73–80. Code: B12.

REFERENCES

- [51] 2017. xMatters Atlassian DevOps Maturity Survey Report 2017. (2017). <https://www.xmatters.com/press-release/xmatters-atlassian-2017-devops-maturity-survey-report/>, accessed on Jun 2018.
- [52] 2018. How Netflix Thinks of DevOps. (2018). <https://www.youtube.com/watch?v=UTKIT6STSVm>, accessed on Jun 2018.
- [53] Nicole Forsgren Velasquez Alanna Brown and, Gene Kim, Nigel Kersten, and Jez Humble. 2016. 2016 State of DevOps Report. (2016). <https://puppet.com/resources/whitepaper/2016-state-of-devops-report>, accessed on Jul 2018.
- [54] Hrishikesh Barua. 2015. The Role of Configuration Management in a Containerized World. (2015). <https://www.infoq.com/news/2015/12/containers-vs-config-mgmt>, accessed on July 2018.
- [55] Len Bass, Ingo Weber, and Liming Zhu. 2015. *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional.
- [56] Helen Beal. 2015. Where are you on the DevOps Maturity Scale Webcast. (2015). <https://www.youtube.com/watch?v=a50ArHzVRqk>, accessed on Jul 2018.
- [57] Kent Beck and Cynthia Andres. 2004. *Extreme programming explained: embrace change*. Addison-Wesley Professional.
- [58] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.
- [59] Jonas Bonér, Dave Farley, Roland Kuhn, and Martin Thompson. 2014. The Reactive Manifesto. (2014). <https://www.reactivemanifesto.org/>, accessed on August 2018.
- [60] Rob Brigham. 2015. DevOps at Amazon: A Look at Our Tools and Processes. (2015). At AWS re:Invent 2015, <https://www.youtube.com/watch?v=esEFaY0FDKc>, accessed on Jun 2018.
- [61] Donovan Brown. 2018. Our DevOps journey - Microsoft's internal transformation story. (2018). DevOneConf 2018, <https://www.youtube.com/watch?v=cbFz0jQOjyA>, accessed on Jul 2018.
- [62] David Budgen and Pearl Brereton. 2006. Performing Systematic Literature Reviews in Software Engineering. In *Proceedings of the 28th International Conference on Software Engineering (ICSE '06)*. ACM, 1051–1052.
- [63] Necco Ceresani. 2016. The Periodic Table of DevOps Tools v.2 Is Here. (2016). <https://blog.xebialabs.com/2016/06/14/periodic-table-devops-tools-v-2/>, accessed on April 2018.
- [64] Kathy Charmaz. 2008. Chapter 7: Grounded Theory as an Emergent Method. In *Handbook of Emergent Methods*. The Guilford Press.
- [65] Gerry Coleman and Rory O'Connor. 2008. Investigating software process in practice: A grounded theory perspective. *Journal of Systems and Software* 81, 5 (2008), 772–784.
- [66] Juliet Corbin and Anselm Strauss. 2014. *Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory* (4th ed.). SAGE Publications, Inc.
- [67] Breno B. Nicolau de França, Helvio Jeronimo, Junior, and Guilherme Horta Travassos. 2016. Characterizing DevOps by Hearing Multiple Voices. In *Proceedings of the 30th Brazilian Symposium on Software Engineering (SBES '16)*. ACM, 53–62.
- [68] Patrick Debois. 2008. Just Enough Documented Information. (2008). At Agile 2008 Toronto.

- [69] Phil Dougherty. 2015. Containers Vs. Config Management. (2015). <https://blog.containership.io/containers-vs-config-management-e64cbb744a94>, accessed on July 2018.
- [70] Floris Erich, Chintan Amrit, and Maya Daneva. 2014. A Mapping Study on Cooperation between Information System Development and Operations. In *Product-Focused Software Process Improvement*, Andreas Jedlitschka, Pasi Kuvaja, Marco Kuhrmann, Tomi Männistö, Jürgen Münch, and Mikko Raatikainen (Eds.). Springer International Publishing, Cham, 277–280.
- [71] F. M. A. Erich, C. Amrit, and M. Daneva. 2017. A Qualitative Study of DevOps Usage in Practice. *Journal of Software: Evolution and Process* 29, 6 (2017), e1885.
- [72] Nicole Forsgren, Jez Humble, and Gene Kim. 2018. *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. IT Revolution Press.
- [73] Martin Fowler. 2004. StranglerApplication. (2004). <https://www.martinfowler.com/bliki/StranglerApplication.html>, accessed on Jul 2018.
- [74] Martin Fowler. 2010. Blue Green Deployment. (2010). <https://martinfowler.com/bliki/BlueGreenDeployment.html>, accessed on Jul 2018.
- [75] Georges Bou Ghantous and Asif Gill. 2017. DevOps: Concepts, Practices, Tools, Benefits and Challenges. In *21st Pacific Asia Conference on Information Systems (PACIS 2017)*. 96:1–96:12.
- [76] David Guest. 1991. The hunt is on for the Renaissance Man of computing. *The Independent* (London). (September 1991).
- [77] Peter J Hager, Howard Jeffrey Scheiber, and Nancy C Corbin. 1997. *Designing & delivering: Scientific, technical, and managerial presentations*. John Wiley & Sons.
- [78] James Hamilton. 2007. On Designing and Deploying Internet-Scale Services. In *Proceedings of the 21st Large Installation System Administration Conference (LISA '07)*. USENIX, 231–242.
- [79] Pete Hodgson. 2017. Feature Toggles (aka Feature Flags). (2017). <https://martinfowler.com/articles/feature-toggles.html>, accessed on Jul 2018.
- [80] Jonah Horowitz. 2017. Configuration Management is an Antipattern. (2017). <https://hackernoon.com/configuration-management-is-an-antipattern-e677e34be64c>, accessed on July 2018.
- [81] Jez Humble. 2010. Continuous Delivery vs Continuous Deployment. (2010). <https://continuousdelivery.com/2010/08/continuous-delivery-vs-continuous-deployment/>, accessed on April 2018.
- [82] Jez Humble. 2012. There's No Such Thing as a "Devops Team". (2012). <https://continuousdelivery.com/2012/10/theres-no-such-thing-as-a-devops-team/>, accessed on May 2018.
- [83] Jez Humble and David Farley. 2010. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
- [84] Ramtin Jabbari, Nauman bin Ali, Kai Petersen, and Binish Tanveer. 2016. What is DevOps?: A Systematic Mapping Study on Definitions and Practices. In *Proceedings of the Scientific Workshop Proceedings of XP2016 (XP '16 Workshops)*. ACM, 12:1–12:11.
- [85] Adam Jacob. 2015. Chef Style DevOps Kungfu. (2015). At ChefConf 2015, https://www.youtube.com/watch?v=_DEToXsgrPc, accessed on Jun 2018.
- [86] Dan Kelly. 2016. Configuration Management And Containers: Which Is Better? (2016). <https://blog.containership.io/configuration-management-and-containers-which-is-better>, accessed on July 2018.
- [87] N. Kerzazi and B. Adams. 2016. Botched Releases: Do We Need to Roll Back? Empirical Study on a Commercial Web App. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, Vol. 1. 574–583.
- [88] N. Kerzazi and B. Adams. 2016. Who Needs Release and DevOps Engineers, and Why?. In *2016 IEEE/ACM International Workshop on Continuous Software Evolution and Delivery (CSED)*. 77–83.
- [89] Gene Kim. 2012. The Three Ways: The Principles Underpinning DevOps. (2012). <http://itrevolution.com/the-three-ways-principles-underpinning-devops/>, accessed on Jul 2018.
- [90] Gene Kim, Kevin Behr, and Kim Spafford. 2014. *The phoenix project: A novel about IT, DevOps, and helping your business win*. IT Revolution.
- [91] Gene Kim, Jez Humble, Patrick Debois, and John Willis. 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
- [92] Henrik Kniberg. 2014. Spotify engineering culture (part 1). (2014). <https://labs.spotify.com/2014/03/27/spotify-engineering-culture-part-1>, accessed on Sep 2018.
- [93] Per Kroll and Philippe Kruchten. 2003. *The rational unified process made easy: a practitioner's guide to the RUP*. Addison-Wesley Professional.
- [94] Patrick Kua. 2013. An Appropriate Use of Metrics. (2013). <https://martinfowler.com/articles/useOfMetrics.html>, accessed on Jul 2018.

- [95] James Lewis and Martin Fowler. 2014. Microservices. (2014). <https://www.martinfowler.com/articles/microservices.html>, accessed on Jul 2018.
- [96] Lucy Ellen Lwakatare, Pasi Kuvaja, and Markku Oivo. 2015. Dimensions of DevOps. In *Agile Processes in Software Engineering and Extreme Programming*. Springer International Publishing, 212–217.
- [97] Robert C. Martin. 2008. Chapter 12: Emergence. In *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- [98] M. Douglas McIlroy, J. M. Buxton, Peter Naur, and Brian Randell. 1968. Mass-produced software components. In *Software Engineering Concepts and Techniques, 1968 NATO Conference on Software Engineering*. 88–98.
- [99] Peter Mell and Timothy Grance. 2011. The NIST definition of cloud computing. (2011). <http://csrc.nist.gov/publications/nistpubs/800-145/SP800-145.pdf>, accessed on May 2018.
- [100] Matthew B. Miles and A. Michael Huberman. 1994. Chapter 2: Focusing and Bounding the Collection of Data - the Substantive Start. In *Qualitative Data Analysis: An Expanded Sourcebook* (6th ed.). SAGE Publications.
- [101] Dejan Milojicic. 2011. Autograding in the Cloud: Interview with David O'Hallaron. *IEEE Internet Computing* 15, 1 (2011), 9–12.
- [102] Kief Morris. 2016. *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media.
- [103] Eueung Mulyana, Rifqy Hakimi, and Hendrawan. 2018. Bringing Automation to the Classroom: A ChatOps-Based Approach. In *2018 4th International Conference on Wireless and Telematics (ICWT)*. 1–6.
- [104] Michael T. Nygard. 2009. *Release It! Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.
- [105] Mary Poppendieck and Tom Poppendieck. 2006. *Implementing Lean Software Development: From Concept to Cash*. Addison-Wesley Professional.
- [106] Roger S Pressman. 2005. *Software engineering: a practitioner's approach* (6th ed.). Palgrave Macmillan.
- [107] Mike Roberts. 2018. Serverless Architectures. (2018). <https://martinfowler.com/articles/serverless.html>, accessed on Oct 2018.
- [108] Kevin Roebuck. 2011. *DevOps: High-impact Strategies - What You Need to Know: Definitions, Adoptions, Impact, Benefits, Maturity, Vendors*. Tebbo.
- [109] Margaret Rouse. 2015. What is NoOps? - Definition from WhatIs.com. (2015). <https://searchcloudapplications.techtarget.com/definition/noops>, accessed on May 2018.
- [110] Danilo Sato. 2014. CanaryRelease. (2014). <https://martinfowler.com/bliki/CanaryRelease.html>, accessed on Jul 2018.
- [111] Danilo Sato. 2014. Chapter 12: Infrastructure as Code. In *Devops in Practice: Reliable and automated software delivery*. Casa do Código.
- [112] Alexandra Sbaraini, Stacy M Carter, R Wendell Evans, and Anthony Blinkhorn. 2011. How to do a grounded theory study: a worked example of a study of dental practices. *BMC medical research methodology* 11, 128 (2011), 1–20.
- [113] Julia Silge. 2017. How Much Do Developers Earn? Find Out with the Stack Overflow Salary Calculator. (2017). <https://stackoverflow.blog/2017/09/19/much-developers-earn-find-stack-overflow-salary-calculator/>, accessed on April 2018.
- [114] Matthew Skelton and Manuel Pais. 2013. DevOps Topologies. (2013). <https://web.devopstopologies.com/>, accessed on Jul 2018.
- [115] Jens Smeds, Kristian Nybom, and Ivan Porres. 2015. DevOps: A Definition and Perceived Adoption Impediments. In *Agile Processes in Software Engineering and Extreme Programming*. Springer International Publishing, 166–177.
- [116] Ian Sommerville. 2011. *Software engineering* (9th ed.). Addison-Wesley.
- [117] Daniel Stahl, Torvald Martensson, and Jan Bosch. 2017. Continuous practices and devops: beyond the buzz, what does it all mean?. In *43rd Euromicro Conference on Software Engineering and Advanced Applications (SEAA 2017)*. 440–448.
- [118] Klaas-Jan Stol, Paul Ralph, and Brian Fitzgerald. 2016. Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE '16)*. 120–131.
- [119] Antonio Terceiro, Joenio Costa, João Miranda, Paulo Meirelles, Luiz Romário Rios, Lucianna Almeida, Christina Chavez, and Fabio Kon. 2010. Analizo: an extensible multi-language source code analysis and visualization toolkit. In *Brazilian Conference on Software: Theory and Practice (Tools Session) (CBSOFT)*, Vol. 29.
- [120] JC van Winkel. 2017. Life of an SRE at Google. (2017). At Codemotion Rome 2017, <https://www.youtube.com/watch?v=7Oe8mYPBZmw>, accessed on Jun 2018.
- [121] Nicole Forsgren Velasquez, Gene Kim, Nigel Kersten, and Jez Humble. 2014. 2014 State of DevOps Report. (2014). <https://puppet.com/resources/whitepaper/2014-state-devops-report>, accessed on May 2018.
- [122] Adam Wiggins. 2011. The Twelve-Factor App. (2011). <https://12factor.net/>, accessed on August 2018.
- [123] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE '14)*. ACM, 38:1–38:10.